

Chủ đề 5

BÀI 16

Giải quyết vấn đề với sự trợ giúp của máy tính

NGÔN NGỮ LẬP TRÌNH BẬC CAO VÀ PYTHON

SAU BÀI NÀY EM SẼ:

- Biết khái niệm ngôn ngữ lập trình bậc cao và ngôn ngữ lập trình bậc cao Python.
- Phân biệt được chế độ gõ lệnh trực tiếp và chế độ soạn thảo chương trình trong môi trường lập trình Python.
- Biết cách tạo và thực hiện một chương trình Python.



Em hãy quan sát các đoạn chương trình được viết bằng các ngôn ngữ lập trình khác nhau trong Hình 16.1 và cho biết câu lệnh trong ngôn ngữ nào dễ hiểu nhất.

```
01101101 01101001 01101110 00100000
01101110 00100000 01101100 01110010
11100001 10111010 10100001 01101110
10100101 01110100 00100000 11000100
00100000 01110100 01110010 01100001
```

Ngôn ngữ máy

0013	RESETA	EQU	%00010011
0011	CTLREG	EQU	%00010001
C003	86	13	INITA LDA A #RESETA
C005	B7	80	04 STA A ACIA
C008	86	11	LDA A #CTLREG
C00D	7E	80	04 STA A ACIA
C00D	7E	C0	F1 JMP SIGNON

Hợp ngữ (Assembly)

```
# Nhập từ bàn phím dãy n số và tính tổng.
n = int(input("Nhập số tự nhiên n: "))
s = 0
for i in range(n):
    a = int(input("Nhập số thứ "+str(i+1)+":"))
    s = s + a
print("Tổng các số đã nhập là:",s)
```

Ngôn ngữ Python

Hình 16.1

1. NGÔN NGỮ LẬP TRÌNH BẬC CAO

Hoạt động 1 Tìm hiểu ngôn ngữ lập trình bậc cao

1. Ngôn ngữ lập trình là gì? Có những loại ngôn ngữ lập trình nào?
2. Hãy kể tên một số ngôn ngữ lập trình bậc cao mà em biết.



Các lệnh viết bằng **ngôn ngữ máy** ở dạng mã nhị phân hay **hợp ngữ** sử dụng một số từ viết tắt (thường là tiếng Anh) không thuận tiện cho việc viết hoặc hiểu chương trình. Khó khăn đó đã được giảm bớt khi lập trình bằng **ngôn ngữ bậc cao** với các câu lệnh được viết gần với ngôn ngữ tự nhiên. Tuy nhiên, để máy tính có thể hiểu và thực hiện, các chương trình đó cần được dịch sang ngôn ngữ máy nhờ một chương trình chuyên dụng được gọi là **chương trình dịch**.

Hiện nay đã có nhiều ngôn ngữ lập trình bậc cao khác nhau, trong số đó Java, C/C++, Python,... là những ngôn ngữ lập trình thông dụng nhất.

Python là ngôn ngữ lập trình bậc cao do Guido van Rossum, người Hà Lan tạo ra và ra mắt lần đầu năm 1991. Các câu lệnh của Python có cú pháp đơn giản. Môi trường lập trình Python dễ sử dụng, không phụ thuộc vào hệ điều hành, chạy trên nhiều loại máy tính, điện thoại thông minh, robot giáo dục,... Python có mã nguồn mở nên thu hút nhiều nhà khoa học cùng phát triển. Nhờ có các thư viện chương trình phong phú về trí tuệ nhân tạo, phân tích dữ liệu, kĩ thuật robot,... Python là ngôn ngữ lập trình được dùng phổ biến trong nghiên cứu và giáo dục.

- *Ngôn ngữ lập trình bậc cao có các câu lệnh được viết gần với ngôn ngữ tự nhiên giúp cho việc đọc, hiểu chương trình dễ dàng hơn.*
- *Python là một ngôn ngữ lập trình bậc cao phổ biến trong nghiên cứu và giáo dục.*



Theo em, viết chương trình bằng loại ngôn ngữ lập trình nào dễ nhất?

- A. Ngôn ngữ máy. B. Hợp ngữ. C. Ngôn ngữ lập trình bậc cao.

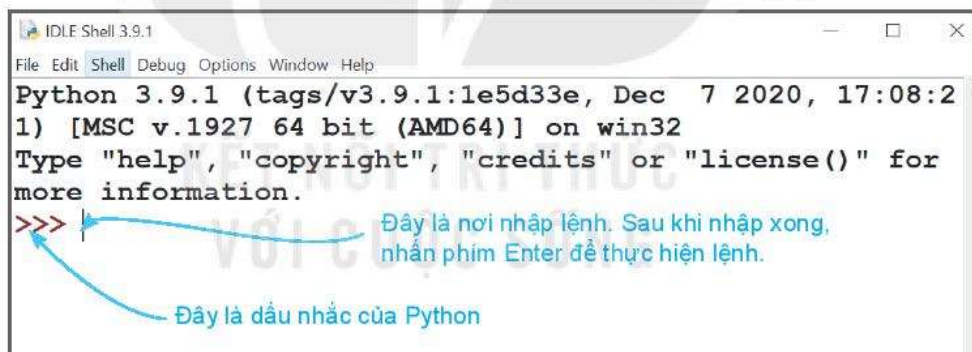
2. MÔI TRƯỜNG LẬP TRÌNH PYTHON

Hoạt động 2 Làm quen với môi trường lập trình Python

1. Tìm hiểu cách viết và thực hiện các lệnh trong môi trường lập trình Python.
2. Phân biệt chế độ gõ lệnh trực tiếp và chế độ soạn thảo chương trình của Python.



Sau khi khởi động, màn hình làm việc của Python có dạng tương tự như sau:



Hình 16.2. Màn hình làm việc của Python

Phần mềm Python là một môi trường lập trình cho phép soạn thảo chương trình bằng ngôn ngữ Python, hỗ trợ gỡ lỗi, phân tích cú pháp dòng lệnh và thực hiện các chương trình Python (chương trình hoàn chỉnh hoặc từng câu lệnh). Môi trường lập trình Python có hai chế độ:

- Chế độ gõ lệnh trực tiếp thường được dùng để tính toán và kiểm tra nhanh các lệnh.
- Chế độ soạn thảo dùng để viết các chương trình có nhiều dòng lệnh.

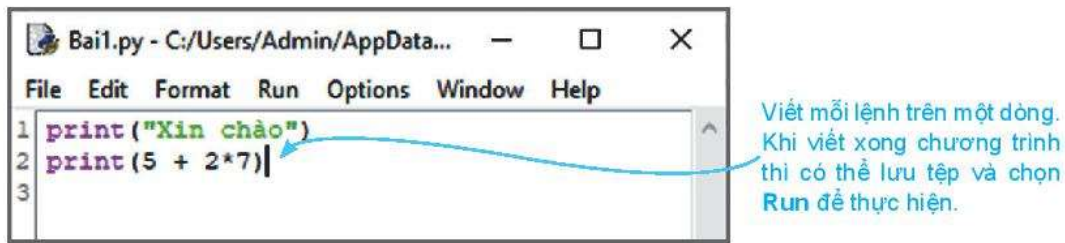
a) Chế độ gõ lệnh trực tiếp

Trong một phiên làm việc với Python, em có thể gõ lệnh trực tiếp sau dấu nhắc `>>>` và nhấn phím Enter để thực hiện lệnh như sau:

```
>>> <lệnh Python>
```

b) Chế độ soạn thảo

Trong môi trường lập trình Python, chúng ta cũng có thể soạn thảo chương trình hoàn chỉnh bằng cách chọn **File/New File** để mở ra màn hình soạn thảo chương trình tương tự như sau:



Hình 16.3. Màn hình soạn thảo trong môi trường Python

Chú ý. Người ta có thể soạn thảo chương trình Python bằng phần mềm soạn thảo văn bản hoặc phần mềm lập trình Python như Wingware, PyCharm, Thonny, Visual Studio,...

Môi trường lập trình của Python có hai chế độ: chế độ gõ lệnh trực tiếp và chế độ soạn thảo.



1. Dấu nhắc chính là con trỏ soạn thảo chương trình Python. Đúng hay sai?
2. Việc thực hiện câu lệnh ở chế độ gõ lệnh trực tiếp và chế độ soạn thảo có điểm gì giống nhau, khác nhau?

3. MỘT SỐ LỆNH PYTHON ĐẦU TIÊN

Hoạt động 3 Làm quen với câu lệnh của Python

Quan sát một số lệnh trong chế độ gõ lệnh trực tiếp để biết chức năng của các lệnh này.



Ví dụ 1. Các lệnh đầu tiên.

```
>>> 5
```

```
5
```

```
>>> 2.6
```

```
2.6
```

```
>>> "học sinh lớp 10"
```

```
'học sinh lớp 10'
```

Khi nhập 5, Python hiểu rằng đó là số nguyên 5.
Nếu nhập 2.6 thì Python tự động nhận ra đó là số thực 2.6.
Còn nếu gõ các chữ giữa cặp dấu nháy kép " " thì Python hiểu là xâu kí tự.

Python tự nhận biết kiểu dữ liệu và thực hiện các phép toán ngay trên dòng lệnh.

Ví dụ 2. Các lệnh với phép toán.

```
>>> 3 + 7
```

```
10
```

```
>>> 12*5
```

```
60
```

Các phép toán thông thường với số bao gồm phép cộng (+), trừ (-), nhân (*) và chia (/).

Ví dụ 3. Lệnh print().

```
>>> print(12)
12
>>> print(10, 3.4 + 4.1, "hoà bình")
10 7.5 hoà bình
>>> print("Dãy ba số chẵn:", 2, 4, 6)
Dãy ba số chẵn: 2 4 6
>>> print("3 + 7 =", 3+7)
3 + 7 = 10
```

Chú ý: nếu lệnh print() nhiều giá trị thì các dữ liệu này sẽ được đưa ra trên một dòng, giữa các dữ liệu sẽ có dấu cách.

Lệnh print() có thể tính toán và đưa ra kết quả của biểu thức.

Trong Python, lệnh print() có chức năng đưa dữ liệu ra (xuất dữ liệu). Mặc định dữ liệu sẽ được in ra màn hình. Lệnh print() cho phép in một hoặc nhiều giá trị ra màn hình. Cú pháp lệnh print() như sau:

```
print(v1, v2, ..., vn)
```

trong đó v1, v2, ..., vn là các giá trị cần đưa ra màn hình.

- Khi nhập giá trị số hoặc xâu kí tự từ dòng lệnh, Python tự nhận biết kiểu dữ liệu.
- Python có thể thực hiện các phép toán thông thường với số, phân biệt số thực và số nguyên.
- Lệnh print() có chức năng in dữ liệu ra màn hình, có thể in ra một hoặc nhiều giá trị đồng thời.



1. Kết quả của mỗi lệnh sau là gì? Kết quả đó có kiểu dữ liệu nào?

```
>>> 5/2
>>> 12 + 1.5
>>> "Bạn là học sinh lớp 10"
>>> 10 + 7/2
```

2. Lệnh sau sẽ in ra kết quả gì?

```
>>> print("13 + 10*3/2 - 3*2 = ", 13 + 10*3/2 - 3*2)
```



THỰC HÀNH

Nhiệm vụ: Sử dụng chế độ soạn thảo chương trình của Python để tạo, nhập và chạy chương trình đầu tiên có tên **Bai1.py** như sau:

Bai1.py

```
# Chương trình đầu tiên
print("Xin chào!")
```

Kí hiệu # là vị trí bắt đầu dòng chú thích lệnh của Python.

Hướng dẫn.

Bước 1: Nháy đúp chuột vào biểu tượng  của Python để khởi động.

Bước 2: Chọn chế độ soạn thảo chương trình của môi trường lập trình Python: Trong môi trường lập trình Python, chọn **File/New**.

Bước 3: Nhập nội dung chương trình như Hình 16.4.



Hình 16.4

Bước 4: Chọn **File/Save** hoặc nhấn tổ hợp phím **Ctrl + S** để lưu tệp.

Bước 5: Chọn **Run/Run module** hoặc nhấn phím **F5** để thực hiện chương trình.

Bước 6: Để kết thúc một phiên làm việc, nháy nút  ở góc trên bên phải màn hình hoặc gõ lệnh `quit()` hoặc lệnh `exit()` rồi nhấn phím ENTER. Ví dụ:

```
>>> quit()
```



LUYỆN TẬP

1. Hãy viết lệnh để tính giá trị các biểu thức sau trong chế độ gõ lệnh trực tiếp của Python:

a) $10 + 13$

b) $20 - 7$

c) $3 \times 10 - 16$

d) $12/5 + 13/6$

2. Các lệnh sau có lỗi không? Vì sao?

```
>>> 3 + * 5
```

```
>>> "Bạn là học sinh, bạn tên là "Nguyễn Việt Anh""
```

3. Viết các lệnh in ra màn hình thông tin như sau:

a) $1 \times 3 \times 5 \times 7 = 105$.

b) Bạn Hoa năm nay 16 tuổi.



VẬN DỤNG

1. Ngoài cách viết chuỗi kí tự giữa cặp dấu nháy đơn hoặc nháy kép còn có thể viết giữa cặp ba dấu nháy kép. Nếu một chuỗi được viết giữa cặp ba dấu nháy kép thì chúng ta có thể dùng phím **Enter** để xuống dòng ở giữa chuỗi. Hãy thực hiện lệnh sau và quan sát kết quả:

```
>>> print("""Không có việc gì khó  
Chỉ sợ lòng không bền  
Đào núi và lấp biển  
Quyết chí ắt làm nên""")
```

2. Viết chương trình Python in ra màn hình bảng nhân trong phạm vi 10.

SAU BÀI NÀY EM SẼ:

- Biết cách thiết lập biến. Phân biệt được biến và từ khoá.
- Biết sử dụng lệnh gán và thực hiện một số phép toán trên kiểu số nguyên, số thực và xâu kí tự.



Trong Đại số, người ta dùng chữ để thay thế cho số cụ thể, ví dụ hằng đẳng thức $(a + b)^2 = a^2 + 2ab + b^2$ đúng cho mọi giá trị a, b . Trong các ngôn ngữ lập trình, người ta cũng dùng các kí tự hoặc nhóm các kí tự được gọi là *biến* (variable) để thay cho việc phải chỉ ra các giá trị dữ liệu cụ thể.

Theo em, sử dụng biến có những lợi ích gì?

1. BIẾN VÀ LỆNH GÁN

Hoạt động 1 Tìm hiểu khái niệm biến và lệnh gán

Quan sát các lệnh sau, n ở đây được hiểu là gì?

```
>>> n = 5
>>> n
5
>>> n + 3
8
```

Sau khi gán $n = 5$, n sẽ được hiểu là đối tượng số nguyên có giá trị 5.



Biến là tên (định danh) của một vùng nhớ dùng để lưu trữ giá trị (dữ liệu) và giá trị đó có thể được thay đổi khi thực hiện chương trình. Biến trong Python được tạo ra khi thực hiện lệnh gán. Cú pháp của **lệnh gán** như sau:

```
<biến> = <giá trị>
```

Khi thực hiện lệnh gán, <giá trị> bên phải sẽ được gán cho <biến>. Nếu biến chưa được khai báo thì nó sẽ được khởi tạo khi thực hiện câu lệnh gán. Biến trong Python được xác định kiểu dữ liệu tại thời điểm gán giá trị nên không cần khai báo trước kiểu dữ liệu cho biến như một số ngôn ngữ khác. Ví dụ:

```
>>> x = 5
>>> x
5
>>> y = "Tin học 10"
>>> y
'Tin học 10'
```

x được gán 5 và có kiểu số nguyên tại đây.

y được gán "Tin học 10" có kiểu xâu kí tự tại đây.

Sau câu lệnh gán, kiểu dữ liệu của biến được xác định. Cần ghi nhớ điều này để tránh thực hiện các phép toán giữa các biến có kiểu dữ liệu khác nhau. Chẳng hạn, nếu sau các lệnh trên, ta thực hiện lệnh $z = x + y$ thì Python sẽ thông báo lỗi do không thể thực hiện phép cộng giữa dữ liệu kiểu số (biến x) với dữ liệu kiểu chuỗi ký tự (biến y).

Có thể thực hiện tất cả các phép toán thông thường như: $+$, $-$, $*$, $/$,... trên các biến có cùng kiểu dữ liệu. Ví dụ:

```
>>> x = y = 1          ← Có thể gán đồng thời nhiều biến với một giá trị.
>>> x = y + 1
>>> z = (x+y)**x        ← Phép tính lũy thừa: (x + y)x
>>> z
9
```

Ví dụ trên cho thấy có thể gán giá trị biểu thức cho biến. Câu lệnh gán có cú pháp tổng quát như sau:

```
<biến> = <biểu thức>
```

Khi thực hiện lệnh này, Python sẽ tính giá trị của <biểu thức> và gán kết quả cho <biến>. Do đó, mọi biến có trong <biểu thức> đều cần được xác định giá trị trước đó.

Ngoài việc gán giá trị trực tiếp cho biến, ta có thể gán giá trị cho biến thông qua tính toán giá trị của biểu thức với các biến đã được xác định trước. Ví dụ:

```
>>> x = 5                ← x là biến kiểu số nguyên có giá trị bằng 5.
>>> y = 2                ← y là biến kiểu số nguyên có giá trị bằng 2.
>>> z = x/y              ← z là biến kiểu số thực có giá trị bằng 2.5.
>>> z
2.5
```

Tên biến thường được đặt sao cho dễ nhớ và có ý nghĩa.

```
>>> ten = "Hoài Nam"
>>> print("Xin chào",ten)
Xin chào Hoài Nam
```

Có thể gán nhiều giá trị đồng thời cho nhiều biến.

```
>>> x,y,z = 10,5,1
>>> x+y+z
16
```

Cú pháp của lệnh gán đồng thời như sau:

```
<var1>, <var2>, ..., <varn> = <gt1>, <gt2>, ..., <gtn>
```

Chú ý trong lệnh trên, số các biến bên trái bằng số các giá trị bên phải dấu $=$.

- **Biến** là tên của một vùng nhớ dùng để lưu trữ giá trị (dữ liệu) và giá trị đó có thể được thay đổi khi thực hiện chương trình.
- Cú pháp **lệnh gán**:
`<biến> = <biểu thức>`
- Quy tắc đặt tên biến:
 - Chỉ gồm các chữ cái tiếng Anh, các chữ số từ 0 đến 9 và kí tự gạch dưới "_".
 - Không bắt đầu bằng chữ số.
 - Phân biệt chữ hoa và chữ thường.



1. Các tên biến nào dưới đây là hợp lệ trong Python?

- A. `_name` B. `12abc` C. `My country` D. `m123&b` E. `xyzABC`

2. Sau các lệnh dưới đây, các biến x, y nhận giá trị bao nhiêu?

```
>>> x = 10
>>> y = x**2 - 1
>>> x = x/2 + y
```

3. a, b nhận giá trị gì sau các lệnh sau?

```
>>> a,b = 2,3
>>> a,b = a+b, a-b
```

2. CÁC PHÉP TOÁN TRÊN MỘT SỐ KIỂU DỮ LIỆU CƠ BẢN

Hoạt động 2 Các phép toán trên dữ liệu kiểu số và kiểu xâu kí tự

Tìm hiểu các phép toán trên dữ liệu kiểu số và kiểu xâu kí tự.



Ví dụ 1. Các phép toán trên dữ liệu kiểu số.

```
>>> a,b = 10,3
>>> (a+b)**2 + (a-b)*10
239
>>> (a//b)*b + a%b
10
>>> c = b/2
>>> c
1.5
```

Sau lệnh này thì $a = 10$, $b = 3$.

Phép toán `//` lấy thương nguyên của phép chia nguyên.
 Phép toán `%` lấy số dư của phép chia nguyên.

Sau lệnh này, c là biến kiểu thực có giá trị 1.5.

Tất cả các phép toán đều được thực hiện từ trái sang phải, riêng phép lũy thừa (`**`) thì thực hiện từ phải sang trái. Ví dụ, biểu thức `4**2**3` được thực hiện như sau $4^{(2^3)} = 4^8 = 65536$.

```
>>> 4**2**3
```

```
65536
```

```
>>> (4**2)**3
```

```
4096
```

Các phép toán cơ bản với dữ liệu kiểu số (số thực và số nguyên) trong Python là phép cộng "+", trừ "-", nhân "*", chia "/", lấy thương nguyên "//", lấy số dư "%" và phép lũy thừa "**".

Thứ tự thực hiện các phép tính như sau: phép lũy thừa ** có ưu tiên cao nhất, sau đó là các phép toán /, *, //, %, cuối cùng là các phép toán +, -. Ví dụ, lệnh sau:

```
>>> 3/2+4*2**4-5//2**2
```

tương đương với lệnh:

```
>>> 3/2+4*(2**4) - 5//(2**2)
```

Chú ý. Nếu có ngoặc thì biểu thức trong ngoặc được ưu tiên thực hiện trước.

Ví dụ 2. Các phép toán với dữ liệu kiểu chuỗi ký tự.

```
>>> s1 = "Hà Nội"
```

```
>>> s2 = "Việt Nam"
```

```
>>> s1 + s2
```

```
'Hà NộiViệt Nam'
```

← Phép + nối hai chuỗi ký tự.

```
>>> "123"*5
```

```
'123123123123123'
```

← Phép * n lặp n lần chuỗi gốc.

```
>>> s*0
```

```
''
```

← Phép * n với số $n \leq 0$ thì được kết quả là chuỗi rỗng.

Trong biểu thức có cả số thực và số nguyên thì kết quả sẽ có kiểu số thực.

- Các phép toán trên dữ liệu kiểu số: +, -, *, /, //, %, **.
- Các phép toán trên dữ liệu kiểu chuỗi: + (nối chuỗi) và * (lặp).



1. Mỗi lệnh sau là đúng hay sai? Nếu đúng thì cho kết quả là bao nhiêu?

```
>>> (12 - 10//2)**2 - 1
```

```
>>> (13 + 45**2)(30//12 - 5/2)
```

2. Mỗi lệnh sau cho kết quả là chuỗi ký tự như thế nào?

```
>>> ""*20 + "010"
```

```
>>> "10"+"0"*5
```

3. TỪ KHOÁ

Hoạt động 3 Phân biệt biến và từ khoá

Quan sát các lệnh sau, tìm hiểu vì sao Python báo lỗi.

```
>>> if = 12
SyntaxError: invalid syntax
>>> with = "Độ rộng"
SyntaxError: invalid syntax
```



Trong Python, cũng như mọi ngôn ngữ lập trình bậc cao, luôn có một tập hợp các từ tiếng Anh đặc biệt được sử dụng vào mục đích riêng của ngôn ngữ lập trình, được gọi là các **từ khoá** (keyword) của ngôn ngữ lập trình. Khi viết chương trình không được đặt tên biến hay các định danh trùng với từ khoá. Các lệnh trên, do đặt tên biến trùng với các từ khoá **if** và **with** nên bị báo lỗi.

Một số từ khoá trong Python phiên bản 3.x.

False	break	else	if	not	as	from	while	return
None	class	except	import	or	assert	global	with	
True	continue	finally	in	pass	del	lambda	yield	
and	def	for	is	raise	elif	nonlocal	try	

Từ khoá là các từ đặc biệt tham gia vào cấu trúc của ngôn ngữ lập trình. Không được phép đặt tên biến hay các định danh trùng với từ khoá.



Các tên biến sau có hợp lệ không?

- a) `_if` b) `global` c) `nolocal` d) `return` e) `true`



THỰC HÀNH

Tạo và làm việc với biến, tính toán với các kiểu dữ liệu cơ bản trong Python.

Nhiệm vụ 1. Thực hiện các phép tính sau trong môi trường lập trình Python, so sánh kết quả với việc tính biểu thức toán học.

- a) $(1 + 2 + 3 + \dots + 10)^3$.
b) $1/2 + 1/3 + 1/4 + 1/5$.
c) Thực hiện lệnh gán $x = 2$, $y = 5$ rồi tính giá trị biểu thức $(x + y)(x^2 + y^2 - 1)$.
d) Thực hiện lệnh gán $a = 2$, $b = 3$, $c = 4$ rồi tính giá trị biểu thức $(a + b + c)(a + b - c)$.

Hướng dẫn. Các phép tính trên có thể thực hiện trong môi trường lập trình Python như sau:

```
>>> (1+2+3+4+5+6+7+8+9+10)**3
>>> 1/2+1/3+1/4+1/5
>>> x, y = 2, 5
>>> (x+y)*(x**2+y**2-1)
>>> a,b,c = 2,3,4
>>> (a+b+c)*(a+b-c)
```

Nhiệm vụ 2. Gán giá trị cho biến R là bán kính hình tròn rồi viết chương trình tính và in ra kết quả theo mẫu:

Chu vi hình tròn là:

Diện tích hình tròn là:

Hướng dẫn. Soạn thảo chương trình sau trong môi trường lập trình Python:

```
R = 4.5
pi = 3.14
print("Chu vi hình tròn là:",2*R*pi)
print("Diện tích hình tròn là:",pi*R*R)
```

Thực hiện chương trình và kiểm tra kết quả, so sánh với chế độ gõ lệnh trực tiếp.



LUYỆN TẬP

1. Lệnh sau có lỗi gì?

```
>>> x = 1
>>> 123a = x + 1
SyntaxError: invalid syntax
```

2. Lệnh sau sẽ in ra kết quả gì?

```
>>> print("đồ rê mi "*3 + "pha son la si đô "*2)
```



VẬN DỤNG

1. Viết các lệnh để thực hiện việc đổi số giây ss cho trước sang số ngày, giờ, phút, giây, in kết quả ra màn hình.

Ví dụ, nếu ss = 684 500 thì kết quả in ra như sau:

684 500 giây = 7 ngày 22 giờ 8 phút 20 giây.

Gợi ý: Sử dụng các phép toán lấy thương nguyên, lấy số dư và các cách đổi sau:

1 ngày = 86 400 giây; 1 giờ = 3 600 giây; 1 phút = 60 giây.

2. Hãy cho biết trước và sau khi thực hiện các lệnh sau, giá trị các biến x, y là bao nhiêu. Em có nhận xét gì về kết quả nhận được?

```
>>> x, y = 10, 7
>>> x, y = y, x
```

SAU BÀI NÀY EM SẼ:

- Biết và thực hiện được một số lệnh vào, ra đơn giản.
- Thực hiện được một số chuyển đổi dữ liệu giữa các kiểu dữ liệu cơ bản.



Để tương tác với người sử dụng trong khi thực hiện chương trình, các ngôn ngữ lập trình có các câu lệnh để đưa dữ liệu ra màn hình hay nhập dữ liệu vào từ bàn phím. Em đã biết Python có lệnh `print()` dùng để đưa dữ liệu ra màn hình. Để nhập dữ liệu từ bàn phím khi thực hiện chương trình, Python sử dụng câu lệnh `input()`.

Hãy nhập tên đầy đủ và tên viết tắt của bạn vào các ô sau:

Tên đầy đủ:

Tên viết tắt:

Em dự đoán lệnh nhập dữ liệu `input()` có cú pháp và chức năng như thế nào?

1. CÁC LỆNH VÀO RA ĐƠN GIẢN

Trong Bài 1 chúng ta đã biết quá trình xử lý thông tin trong máy tính thực chất cũng là xử lý dữ liệu. Do vậy, trong các ngôn ngữ lập trình bậc cao cần có các câu lệnh tương ứng để nhập, xuất dữ liệu. Trong Python, lệnh `print()` có chức năng đưa dữ liệu ra, còn lệnh `input()` đưa dữ liệu vào.

Hoạt động 1 Tìm hiểu chức năng của lệnh `input()`

Quan sát lệnh sau và trả lời các câu hỏi: Lệnh `input()` cho phép nhập dữ liệu từ đâu? Giá trị được nhập sẽ là số hay chuỗi?

```
>>> input("Nhập một số: ")
Nhập một số: 12
'12'
```



Lệnh `input()` có chức năng nhập dữ liệu từ thiết bị vào chuẩn, thường là bàn phím. Nội dung nhập có thể nhập số, biểu thức hay chuỗi và cho kết quả là một chuỗi kí tự. Cú pháp của lệnh `input()` như sau:

```
<biến> = input(<Dòng thông báo>)
```

Cần nhập một chuỗi kí tự thì có thể dùng lệnh `input()` tương tự như sau:

```
name = input("Nhập họ tên em: ")
print("Xin chào ", name)
```

Lệnh `print()` có chức năng đưa dữ liệu ra thiết bị ra chuẩn, thường là màn hình. Thông tin cần đưa ra có thể bao gồm một hay nhiều dữ liệu với kiểu khác nhau, cho phép cả biểu thức tính toán.

Các lệnh vào ra đơn giản của Python bao gồm lệnh `input()` và lệnh `print()`.

2. CHUYỂN ĐỔI KIỂU DỮ LIỆU CƠ BẢN CỦA PYTHON

Hoạt động 2 Nhận biết kiểu dữ liệu của biến

Chúng ta đã biết một số kiểu dữ liệu cơ bản như số nguyên, số thực và chuỗi ký tự. Trong Python có cách nào để nhận biết được kiểu dữ liệu của biến không?



Quan sát các lệnh sau để biết kiểu dữ liệu của mỗi biến.

```
>>> n,x,s = 10,1.8,"One"  ← Gán n = 10, x = 1.8, s = "One" đồng thời.
>>> n
10
>>> type(n)
<class 'int'>  ← Biến n thuộc kiểu int – số nguyên.
>>> type(x)
<class 'float'>  ← Biến x thuộc kiểu float – số thực.
>>> type(s)
<class 'str'>  ← Biến s thuộc kiểu str – chuỗi ký tự.
```

Kiểu dữ liệu logic cũng là kiểu dữ liệu cơ bản và dữ liệu kiểu này chỉ có hai giá trị là True (đúng) và False (sai). Ví dụ dữ liệu kiểu logic là kết quả phép so sánh:

```
>>> 3 > 2
True
>>> 5 < 0
False
>>> b = 10 > 3
>>> type(b)
<class 'bool'>  ← Tên kiểu dữ liệu logic là bool.
```

Các biểu thức so sánh chỉ nhận giá trị True hoặc False, và có giá trị thuộc kiểu logic.

- Một số kiểu dữ liệu cơ bản của Python bao gồm: `int` (số nguyên), `float` (số thực), `str` (chuỗi ký tự), `bool` (logic).
- Lệnh `type()` dùng để nhận biết kiểu dữ liệu của biến trong Python.



Xác định kiểu và giá trị của các biểu thức sau:

- a) `"15 + 20 - 7"` b) `32 > 45` c) `13 != 8 + 5` d) `1 == 2`

Hoạt động 3 Tìm hiểu cách chuyển đổi giữa các kiểu dữ liệu

1. Có chuyển đổi dữ liệu kiểu này sang kiểu khác được không?
2. Giả sử có biến `s` với giá trị `"123"`. Nếu muốn biến `s` có giá trị là số nguyên 123 chứ không phải là chuỗi `"123"` thì em phải làm gì?



Lệnh `int()` có chức năng chuyển đổi số thực hoặc xâu chứa số nguyên thành số nguyên. Quan sát các lệnh sau:

```
>>> int(12.6)
12
>>> int("123")
123
>>> int("10.35")
Traceback (most recent call last):
  File "<pyshell#21>", line 1, in <module>
    int("10.35")
ValueError: invalid literal for int() with base 10: '10.35'
```

Lệnh `int()` không chuyển đổi được xâu chứa số thực.

Lệnh `float()` dùng để chuyển đổi số nguyên và xâu kí tự thành số thực.

```
>>> float(8)
8.0
>>> float("10.23")
10.23
```

Lệnh `str()` dùng để chuyển đổi các kiểu dữ liệu khác thành xâu kí tự.

```
>>> str(12+34)
'46'
>>> str(12.567)
'12.567'
>>> str(2>3)
'False'
```

Chú ý: Các lệnh `int()`, `float()` chỉ có thể chuyển đổi các xâu ghi giá trị số trực tiếp, không chuyển đổi xâu có công thức, ví dụ:

```
>>> int("12+45")
Traceback (most recent call last):
  File "<pyshell#27>", line 1, in <module>
    int("12+45")
ValueError: invalid literal for int() with base 10: '12+45'
```

- Các lệnh `int()`, `float()`, `str()` có chức năng chuyển đổi dữ liệu từ các kiểu khác tương ứng về kiểu số nguyên, số thực và xâu kí tự.
- Các lệnh `int()`, `float()` không thực hiện xâu là biểu thức toán.



1. Mỗi lệnh sau sẽ trả lại các giá trị nào?

- a) `str(150)` b) `int("1110")` c) `float("15.0")`

2. Lệnh nào sau đây sẽ báo lỗi?

- A. `int("12.0")` B. `float(13+1)` C. `str(17.001)`

Hoạt động 4 Nhập dữ liệu kiểu số nguyên hoặc số thực từ bàn phím

Dữ liệu nhập từ bàn phím bằng lệnh `input()` luôn là xâu kí tự nên muốn nhập dữ liệu đầu vào là số nguyên hay số thực thì phải làm thế nào?



Nếu cần nhập số nguyên thì sau khi nhập giá trị số cần dùng lệnh `int()` để chuyển đổi sang kiểu số nguyên như sau:

```
>>> n = int(input("Nhập số tự nhiên: "))
Nhập số tự nhiên: 13
```

Nếu cần nhập số thực thì sau khi nhập giá trị số cần dùng lệnh `float()` để chuyển đổi sang kiểu số thực như sau:

```
>>> x = float(input("Nhập số thực x: "))
```



Dùng lệnh `x = input("Nhập số x:")` để nhập số cho biến `x` là đúng hay sai?



THỰC HÀNH. Nhập dữ liệu từ bàn phím bằng lệnh `input()`.

Nhiệm vụ 1. Viết chương trình nhập lần lượt ba số tự nhiên `m`, `n`, `p`, sau đó in ra tổng của ba số này.

Hướng dẫn. Cần thực hiện lần lượt ba lệnh nhập các số `m`, `n`, `p`. Chú ý cách nhập số nguyên cần dùng lệnh `int()` để chuyển đổi dữ liệu nhập từ bàn phím. Chương trình có thể viết như sau:

```
m = int(input("Nhập số nguyên m: "))
n = int(input("Nhập số nguyên n: "))
p = int(input("Nhập số nguyên p: "))
print("Tổng ba số đã nhập là", m+n+p)
```

Nhiệm vụ 2. Viết chương trình nhập họ tên, sau đó nhập tuổi của học sinh. Chương trình đưa ra thông báo, ví dụ: Bạn Nguyễn Hoà Bình 15 tuổi.

Hướng dẫn. Cần thực hiện hai lệnh nhập dữ liệu, một lệnh nhập tên học sinh, lệnh thứ hai nhập tuổi, sau đó thông báo ra màn hình. Chú ý khi nhập tuổi cần chuyển đổi dữ liệu.

```
ten = input("Nhập họ tên học sinh: ")
tuoi = int(input("Nhập tuổi: "))
print("Bạn", ten, tuoi, "tuổi")
```



LUYỆN TẬP

1. Những lệnh nào trong các lệnh sau sẽ báo lỗi?

- a) `int("12+45")` b) `float(123.56)` c) `float("123,5.5")`

2. Vì sao khi nhập một số thực cần viết lệnh `float(input())`?



VẬN DỤNG

- Viết chương trình nhập giá trị `ss` là số giây từ bàn phím. Thông báo ra màn hình thời gian `ss` giây này sau khi đổi thành thời gian tính bằng ngày, giờ, phút, giây.
- Viết chương trình nhập ba số thực dương `a`, `b`, `c` và tính chu vi, diện tích của tam giác có độ dài các cạnh là `a`, `b`, `c` (`a`, `b`, `c` > 0 và thoả mãn bất đẳng thức tam giác).

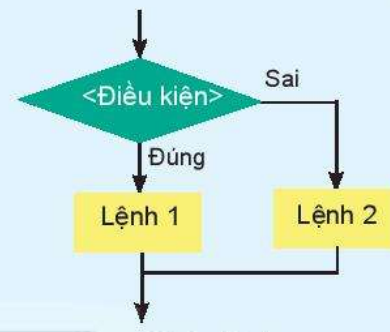
Gợi ý: Công thức Heron tính diện tích tam giác: $S = \sqrt{p(p-a)(p-b)(p-c)}$ với p là nửa chu vi tam giác.

SAU BÀI NÀY EM SẼ:

- Biết và trình bày được các phép toán với kiểu dữ liệu logic.
- Sử dụng được lệnh rẽ nhánh if trong lập trình.



Trong cuộc sống, chúng ta vẫn thường gặp các tình huống một việc được thực hiện hay không phụ thuộc vào một điều kiện. Ví dụ, em dự định, nếu ngày mai trời không mưa em sẽ đi chơi cùng bạn, ngược lại nếu trời mưa em sẽ ở nhà làm bài tập. Các tình huống như vậy trong lập trình được gọi là rẽ nhánh. Em hãy điền thông tin ở tình huống trên vào vị trí <Điều kiện> và lệnh tương ứng trong sơ đồ cấu trúc rẽ nhánh ở Hình 19.1.



Hình 19.1

1. BIỂU THỨC LÔGIC

Hoạt động 1 Khái niệm biểu thức logic

Biểu thức nào sau đây có thể đưa vào vị trí <điều kiện> trong lệnh:

Nếu <điều kiện> thì <lệnh> của các ngôn ngữ lập trình bậc cao?

- A. $m, n = 1, 2$. B. $a + b > 1$. C. $a * b < a + b$. D. $12 + 15 > 2 * 13$.



Trong Python, biểu thức logic là biểu thức chỉ nhận giá trị True (đúng) hoặc False (sai). Biểu thức logic đơn giản nhất là các biểu thức so sánh số hoặc xâu kí tự. Trong Hoạt động 1, các phương án B, C, D là biểu thức logic.

Quan sát các lệnh sau để nhận biết kiểu dữ liệu logic.

```

>>> a,b,s = 10,2,"Number"  ← Gán a = 10, b = 2, s = "Number"
>>> a > 10
False
>>> b < 3
True
>>> s == "number"  ← s và "number" là hai xâu có giá trị khác nhau.
False
  
```

a > 10 là sai, b < 3 là đúng.

Các phép so sánh các giá trị số trong Python:

<	nhỏ hơn	>	lớn hơn	==	bằng nhau
<=	nhỏ hơn hoặc bằng	>=	lớn hơn hoặc bằng	!=	khác nhau

Chú ý: Với xâu kí tự cũng có đầy đủ các phép so sánh (sẽ học sau).

Các phép toán trên kiểu dữ liệu logic bao gồm phép **and** (và), **or** (hoặc) và **not** (phủ định). Bảng các phép toán logic như sau:

Phép toán and		
X	Y	X and Y
True	True	True
True	False	False
False	True	False
False	False	False

Phép toán or		
X	Y	X or Y
True	True	True
True	False	True
False	True	True
False	False	False

Phép toán not	
X	not X
True	False
False	True

Ví dụ. Cho các lệnh sau và dự đoán giá trị của các biến logic a, b, c.

```
>>> x, y, z = 10, 5, 9
>>> b = x < 11 and z > 5
>>> c = x > 15 or y < 9
>>> a = not b
```

Giải thích. Ta có $x = 10$, $z = 9$ do đó $x < 11$ là đúng, $z > 5$ đúng. Theo bảng phép toán **and** ta có $b = x < 11 \text{ and } z > 5$ nhận giá trị đúng.

Ta lại có: $x > 15$ sai (vì $x = 10$) nhưng $y < 9$ đúng (vì $y = 5$). Theo bảng phép toán **or** suy ra $c = x > 15 \text{ or } y < 9$ nhận giá trị đúng.

Cuối cùng, vì b là đúng nên $a = \text{not } b$ sẽ nhận giá trị sai.

- Biểu thức logic là biểu thức chỉ nhận giá trị True hoặc False. Giá trị các biểu thức logic thuộc kiểu bool.
- Các phép toán trên kiểu dữ liệu logic là and (và), or (hoặc) và not (phủ định).



Mỗi biểu thức sau có giá trị True hay False?

- a) $100\%4 == 0$ b) $111//5 != 20 \text{ or } 20\%3 != 0$

2. LỆNH IF

Hoạt động 2 Cấu trúc lệnh if trong Python

Cho trước số tự nhiên n (được gán hoặc nhập từ bàn phím). Đoạn chương trình như sau kiểm tra $n > 0$ thì thông báo "n là số lớn hơn 0".

```
if n > 0:
    print("n là số lớn hơn 0")
```

Em có nhận xét gì về cấu trúc lệnh if? Sau <điều kiện> lệnh if có kí tự gì? Lệnh print() được viết như thế nào?



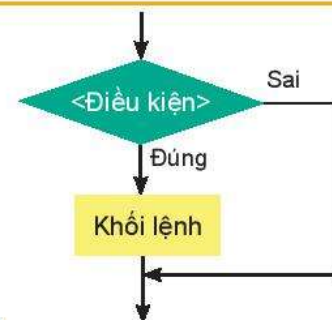
Để xử lí các tình huống rẽ nhánh, giống như các ngôn ngữ lập trình khác, Python cũng có các câu lệnh để mô tả cấu trúc rẽ nhánh:

Câu lệnh rẽ nhánh dạng thiếu:

```
if <điều kiện>:
    <khối lệnh>
```

Sau điều kiện cần có dấu hai chấm ":".

Nhóm, khối lệnh tiếp theo cần viết lùi vào và thẳng hàng, mặc định là 1 tab hay 4 dấu cách.



Hình 19.2

Khi thực hiện lệnh, Python sẽ kiểm tra <điều kiện> nếu đúng thì thực hiện <khối lệnh>, ngược lại thì bỏ qua chuyển sang lệnh tiếp theo sau lệnh `if`.

Câu lệnh rẽ nhánh dạng đủ:

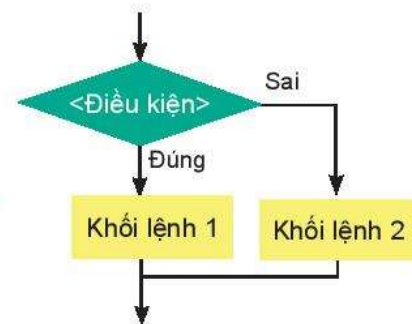
```
if <điều kiện>:  
    <khối lệnh 1>  
else:  
    <khối lệnh 2>
```

- Từ khoá `if` và `else` cần viết thẳng lề trái.
- Các khối lệnh 1 và khối lệnh 2 cần viết lùi vào và thẳng hàng, mặc định là 1 tab hay 4 dấu cách.

Khi thực hiện lệnh, Python sẽ kiểm tra <điều kiện> nếu đúng thì thực hiện <khối lệnh 1>, ngược lại thì thực hiện <khối lệnh 2>.

Ví dụ, nếu `a`, `b` là hai số đã được tạo thì lệnh sau sẽ in ra giá trị tuyệt đối của hiệu hai số:

```
if a > b:  
    print(a - b)  
else:  
    print(b - a)
```



Hình 19.3

Chú ý: Các khối lệnh trong Python đều cần viết sau dấu ":" và lùi vào, thẳng hàng. Đây là điểm khác biệt của Python với các ngôn ngữ lập trình khác.

Câu lệnh rẽ nhánh `if` thể hiện cấu trúc rẽ nhánh trong Python. Khối lệnh rẽ nhánh của `if` được viết sau dấu ":", cần viết lùi vào và thẳng hàng.



Đoạn chương trình sau thực hiện công việc gì?

```
k = int(input("Nhập một số nguyên dương: "))  
if k <= 0:  
    print("Bạn nhập sai rồi!")
```



THỰC HÀNH

Các bài tập liên quan đến kiểu dữ liệu `bool` và lệnh `if`.

Nhiệm vụ 1. Viết chương trình nhập số tự nhiên `n` từ bàn phím. Sau đó thông báo số em đã nhập là số chẵn hay số lẻ phụ thuộc vào `n` là chẵn hay lẻ.

Hướng dẫn. Để kiểm tra một số tự nhiên `n` là chẵn hay lẻ, ta dùng phép toán lấy số dư `n%2`. Nếu số dư bằng 0 thì `n` là số chẵn, ngược lại `n` là số lẻ. Chương trình có thể như sau:

```
n = int(input("Nhập số tự nhiên n: "))  
if n%2 == 0:  
    print("Số đã nhập là số chẵn.")  
else:  
    print("Số đã nhập là số lẻ.")
```

Nhiệm vụ 2. Giả sử giá điện sinh hoạt trong khu vực gia đình em ở được tính lũy kế theo từng tháng như sau (giá tính theo từng kWh điện tiêu thụ).

- Với mức điện tiêu thụ từ 0 đến 50 kWh, giá thành mỗi kWh là 1,678 nghìn đồng.
- Với mức từ 51 đến 100, giá thành mỗi kWh là 1,734 nghìn đồng.
- Từ mức 101 trở lên, giá thành mỗi kWh là 2,014 nghìn đồng.

Viết chương trình nhập số điện tiêu thụ trong tháng của gia đình em và tính số tiền điện phải trả.

Hướng dẫn. Gọi k là số kWh điện tiêu thụ của gia đình em. Khi đó theo cách tính lũy kế trên chúng ta cần tính dựa trên các điều kiện sau:

- Nếu $k \leq 50$ thì số tiền cần trả là $k \times 1,678$ nghìn đồng.
- Nếu $50 < k \leq 100$ thì số tiền cần trả là $50 \times 1,678 + (k - 50) \times 1,734$ nghìn đồng.
- Nếu $100 < k$ thì số tiền cần trả là: $50 \times 1,678 + 50 \times 1,734 + (k - 100) \times 2,014$ nghìn đồng.

Chúng ta sử dụng lệnh `round(t)` để làm tròn số thực t . Chú ý trong máy tính dùng dấu "." để viết các số thập phân. Chương trình có thể như sau:

```
k = float(input("Nhập số kWh tiêu thụ điện nhà em: "))
if k <= 50:
    t = k*1.678
else:
    if k <= 100:
        t = 50*1.678 + (k-50)*1.734
    else:
        t = 50*1.678 + 50*1.734 + (k-100)*2.014
print("Số tiền điện phải trả là:", round(t), "nghìn đồng")
```



LUYỆN TẬP

1. Viết biểu thức logic ứng với mỗi câu sau:

- Số x nằm trong khoảng $(0; 10)$.
- Số y nằm ngoài đoạn $[1; 2]$.
- Số z nằm trong đoạn $[0; 1]$ hoặc $[5; 10]$.

2. Tìm một vài giá trị m, n thỏa mãn các biểu thức sau:

- $100\%m == 0$ and $n\%5 != 0$
- $m\%100 == 0$ and $m\%400 != 0$
- $n\%3 == 0$ or $(n\%3 != 0$ and $n\%4 == 0)$



VẬN DỤNG

- Giá bán cam tại siêu thị tính như sau: nếu khối lượng cam mua dưới 5 kg thì giá bán là 12 000 đồng/kg, nếu khối lượng mua lớn hơn hoặc bằng 5 kg thì giá bán là 10 000 đồng/kg. Viết chương trình nhập số lượng mua (tính theo kg) sau đó tính số tiền phải trả.
- Năm n là năm nhuận nếu giá trị n thỏa mãn điều kiện: n chia hết cho 400 hoặc n chia hết cho 4 đồng thời không chia hết cho 100. Viết chương trình nhập số năm n và cho biết năm n có phải là nhuận hay không.

SAU BÀI NÀY EM SẼ:

- Biết được ý nghĩa của vùng giá trị tạo bởi lệnh `range()`.
- Biết được chức năng của lệnh lặp `for` và cách dùng trong Python.



Em có thể đã gặp những trường hợp cần thực hiện một số công việc lặp đi lặp lại nhiều lần. Ví dụ, để kể tên tất cả các bạn trong lớp có 30 học sinh, em cần lần lượt đọc tên từng bạn; để đếm số lượng các số chia hết cho 3 trong khoảng từ 1 đến 50, em có thể kiểm tra lần lượt các số từ 1 đến 50 và ghi ra các số chia hết cho 3 (chẳng hạn, 3, 6, 9,...) rồi đếm các số đó. Ngôn ngữ lập trình bậc cao có các câu lệnh cho phép viết một cách ngắn gọn các bước cần thực hiện lặp đi lặp lại để tạo thành một cấu trúc lập trình được gọi là cấu trúc lặp.

Em có thể xác định được trong mỗi ví dụ trên công việc nào cần phải lặp và được lặp lại bao nhiêu lần không?

1. LỆNH LẶP FOR

Hoạt động 1 Làm quen với lệnh lặp for

Thực hiện đoạn chương trình sau trong chế độ gõ lệnh trực tiếp của Python để tính tổng $0 + 1 + \dots + 9$. Tổng này có giá trị bao nhiêu? Giải thích kết quả.

```
>>> S = 0
>>> for k in range(10):
    S = S + k
>>> print(S)
45
```

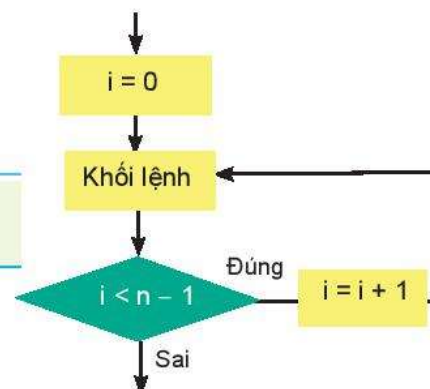


Trong đoạn chương trình trên, lệnh `range(10)` trả lại một *vùng giá trị* gồm 10 số 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Lệnh `for` sẽ thực hiện 10 lần lặp, mỗi lần lặp ứng với một giá trị `k` trong vùng giá trị trên. Sau lệnh lặp `for` trên, biến `S` sẽ có giá trị là tổng $0 + 1 + \dots + 9 = 45$.

Lệnh `range(n)` trả lại *vùng giá trị* gồm `n` số từ 0 đến `n - 1`. Cú pháp của lệnh lặp với số lần biết trước `for` trong Python như sau:

```
for <i> in range(n):
    <khối lệnh>
```

Khi thực hiện, ở mỗi vòng lặp biến `i` sẽ được gán lần lượt các giá trị trong vùng giá trị của lệnh `range()` và thực hiện `<khối lệnh>`.



Hình 20.1

Ví dụ 1. Tính tổng các số tự nhiên chẵn nhỏ hơn n, với n cho trước (n = 10).

```
n = 10
S = 0
for k in range(n):
    if k%2 == 0: ← Điều kiện k là số chẵn là k%2 = 0
        S = S + k
print(S)
```

Ví dụ 2. Đếm số các số nguyên nhỏ hơn n (n = 20) và là bội của 3.

```
n = 20
C = 0
for k in range(n):
    if k%3 == 0: ← Điều kiện k là bội của 3 là k%3 = 0
        C = C + 1
print(C)
```

for là lệnh lặp với số lần biết trước. Số lần lặp thường được xác định bởi vùng giá trị của lệnh **range()**.



Với giá trị n cho trước, so sánh giá trị S trong đoạn chương trình sau với tổng $1 + 2 + \dots + n$.

```
S = 0
for k in range(1,n+1):
    S = S + k
```

2. LỆNH RANGE

Hoạt động 2 Tìm hiểu vùng giá trị xác định bởi lệnh **range()**

Quan sát các lệnh **for** sau và so sánh kết quả in ra để biết vùng giá trị được xác định bởi lệnh **range()**. Lưu ý, lệnh **print()** có thêm tham số để in bộ dữ liệu theo hàng ngang.

```
>>> for k in range(3,10):
    print(k, end = " ")
3 4 5 6 7 8 9 ← Đây là vùng range(3,10)
>>> for k in range(0,15):
    print(k, end = " ")
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 ← Đây là vùng range(0,15)
```



Lệnh tạo vùng giá trị **range()** có các dạng sau:

range(stop) trả lại vùng giá trị từ 0 đến stop - 1.

range(start, stop) trả lại vùng giá trị từ start đến stop - 1.

Ví dụ, **range(n)** cho vùng gồm các số 0, 1, ..., n - 1.

range(1, n+1) cho vùng gồm các số 1, 2, ..., n.

range(0, 99) cho vùng giá trị gồm các số 0, 1, 2, ..., 98.

range(100, 1) cho vùng rỗng.

Lệnh tạo vùng giá trị có cú pháp `range(start, stop)` trả lại vùng giá trị gồm các số nguyên liên tiếp từ `start` đến `stop - 1`.



Hãy biểu diễn các dãy sau đây bằng lệnh `range()`.

- a) 1, 2, 3, ..., 50. b) 5, 6, 7, 8, 9, 10. c) 0, 1. d) 10.



THỰC HÀNH. Lệnh lặp `for` và lệnh `range()`

Nhiệm vụ 1. Nhập số tự nhiên n từ bàn phím và in ra màn hình dãy các ước số của n theo chiều ngang màn hình. Ví dụ nếu $n = 10$ thì chương trình sẽ in ra dãy số 1, 2, 5, 10.

Hướng dẫn. Các ước số của n là các số tự nhiên k thỏa mãn: $n\%k = 0$. Muốn in các số trên một hàng ngang cần dùng thêm tham số `end = " "` trong lệnh `print()`.

Chương trình có thể như sau:

```
n = int(input("Nhập số tự nhiên n: "))
for k in range(1, n+1):
    if n%k == 0:
        print(k, end = " ")
```

Nhiệm vụ 2. Nhập số tự nhiên n từ bàn phím và đếm số các ước số thực sự của n . Ước số thực sự của n là số tự nhiên $k < n$ và là ước của n .

Hướng dẫn. Tương tự như chương trình ở nhiệm vụ 1, điểm khác là cần đếm số các ước số này và không tính n . Tạo một biến có tên `count` để đếm số các ước số thực sự của n .

```
n = int(input("Nhập số tự nhiên n: "))
count = 0
for k in range(1, n):
    if n%k == 0:
        count = count + 1
print(count)
```



LUYỆN TẬP

1. Đoạn chương trình sau in ra kết quả gì?

```
n = int(input("Nhập số tự nhiên n:"))
S = 0
for k in range(n+1):
    S = S + k
print(S*S)
```

2. Viết đoạn chương trình tính tích $1 \times 2 \times 3 \times \dots \times n$ với n được nhập vào từ bàn phím.



VẬN DỤNG

1. Viết chương trình nhập từ bàn phím số tự nhiên n và in ra kết quả $S = 1 + \frac{1}{2} + \dots + \frac{1}{n}$.

2. Viết chương trình nhập từ bàn phím số tự nhiên n và in ra kết quả là tổng sau:

$$S = 1^3 + 2^3 + \dots + n^3.$$

SAU BÀI NÀY EM SẼ:

- Biết và thực hành giải các bài toán sử dụng lệnh lặp while với số lần không biết trước.
- Biết ba cấu trúc lập trình cơ bản: tuần tự, rẽ nhánh, lặp.



Cho các việc được ghi trong cột A và cột B của bảng sau:

A	B
Vận động viên chạy 20 vòng xung quanh sân vận động.	Vận động viên chạy nhiều vòng xung quanh sân vận động trong thời gian 2 tiếng.
Em làm 5 bài tập thầy cô giao về nhà.	Em làm các bài tập về nhà đến giờ ăn cơm thì dừng lại.
Em đi lấy 15 xô nước giúp mẹ.	Em xách các xô nước giúp mẹ cho đến khi đầy thùng nước.

Đối với mỗi hàng, em hãy cho biết công việc được lặp đi lặp lại là gì? Điều kiện để dừng công việc là gì? Số lần thực hiện việc lặp giữa hai cột có gì khác nhau?

1. LỆNH LẶP WHILE

Hoạt động 1 Làm quen với lệnh lặp while

Quan sát đoạn chương trình sau, giải thích kết quả in ra.

```
>>> S = 0
>>> k = 1
>>> while k < 100:
    S = S + k
    k = k + 7
>>> print(S)
750
```

Điều kiện lặp: nếu <điều kiện> là **False** thì dừng lặp.

Khởi các lệnh lặp được viết lùi vào và thẳng hàng. Sau mỗi vòng lặp, k tăng lên 7.

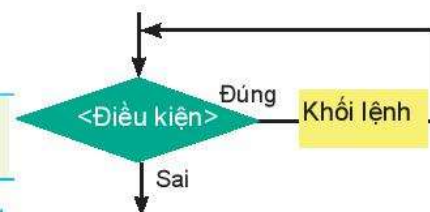


Lệnh lặp **while** thực hiện khối lệnh với số lần lặp không biết trước. Khối lệnh lặp được thực hiện cho đến khi <điều kiện> = False.

Cú pháp của lệnh **while** như sau:

```
while <điều kiện>:
    <khối lệnh>
```

Sau dấu ":", khối lệnh lặp cần được viết lùi vào và thẳng hàng. Mặc định các lệnh sẽ lùi vào 1 tab hoặc 4 dấu cách.



Hình 21.1

Trong đó <điều kiện> là biểu thức logic. Khi thực hiện lệnh, Python sẽ kiểm tra <điều kiện>, nếu đúng thì thực hiện <khối lệnh>, nếu sai thì kết thúc lệnh **while**.

Trong đoạn chương trình ở Hoạt động 1, lệnh lặp sẽ dừng khi $k \geq 100$ và giá trị S nhận được là tổng $1 + 8 + 15 + \dots + 99$.

Ví dụ 1. Quan sát đoạn chương trình sau và cho biết S là giá trị của biểu thức toán học nào?

```
S = 0
k = 1
while k*k < 100:
    S = S + k*k
    k = k + 1
```

Giải thích: Đoạn chương trình tính tổng $1^2 + 2^2 + \dots + k^2$ với điều kiện $k^2 < 100$. Vậy S chính là tổng bình phương các số tự nhiên nhỏ hơn 10.

Ví dụ 2. Thực hiện các lệnh sau. Kết quả sẽ in ra những số nào?

```
>>> k = 2
>>> while k < 50:
    print(k, end = " ")
    k = k + 3
```

Giải thích: Vòng lặp while sẽ dừng khi k vượt quá 50. Bắt đầu vòng lặp, k = 2. Sau mỗi bước lặp k tăng lên 3 đơn vị. Do vậy, kết quả sẽ phải in ra dãy sau:

2 5 8 11 14 17 20 23 26 29 32 35 38 41 44 47

while là lệnh lặp với số lần không biết trước. Số lần lặp của lệnh **while** phụ thuộc vào điều kiện của lệnh.



1. Lệnh **while** kiểm tra điều kiện trước hay sau khi thực hiện khối lệnh lặp?
2. Viết đoạn chương trình tính tổng $2 + 4 + \dots + 100$ sử dụng lệnh **while**.

Lưu ý:

1. Vì lệnh **while** không biết trước số lần lặp, mà phụ thuộc vào điều kiện. Do đó, cần chú ý đến điều kiện của lệnh **while** để tránh bị lặp vô hạn.
2. Trong trường hợp nếu muốn dừng và thoát ngay khỏi vòng lặp **while** hoặc **for** có thể dùng lệnh **break**.

```
>>> for k in range(10):
    print(k, end = " ")
    if k == 5:
        break
```

0 1 2 3 4 5

2. CẤU TRÚC LẬP TRÌNH

Hoạt động 2 Các cấu trúc lập trình cơ bản

Đọc, thảo luận để hiểu các cấu trúc lập trình cơ bản trong ngôn ngữ lập trình bậc cao.



Với việc sử dụng câu lệnh rẽ nhánh **if** và các câu lệnh lặp **for**, **while** ta có thể thấy một chương trình trên Python nói chung có thể được chia thành các khối lệnh sau:

- Khối gồm các lệnh được thực hiện theo trình tự từ trên xuống dưới. Khối này tương ứng với cấu trúc tuần tự trong chương trình và được thể hiện bằng các câu lệnh như gán giá trị, nhập/xuất dữ liệu,...

- Khối các câu lệnh chỉ được thực hiện tùy thuộc vào điều kiện nào đó là đúng hay sai. Khối lệnh này tương ứng với cấu trúc rẽ nhánh và được thể hiện bằng câu lệnh rẽ nhánh `if`.
- Khối các câu lệnh được thực hiện lặp đi lặp lại tùy theo điều kiện nào đó vẫn còn đúng hay sai. Khối lệnh này tương ứng với cấu trúc lặp và được thể hiện bằng các câu lệnh lặp `for`, `while`.

Ba cấu trúc chương trình trên được gọi là các cấu trúc lập trình cơ bản của các ngôn ngữ lập trình bậc cao.

Ba cấu trúc lập trình cơ bản của các ngôn ngữ lập trình bậc cao gồm: cấu trúc tuần tự, cấu trúc rẽ nhánh, cấu trúc lặp.



THỰC HÀNH. Sử dụng lệnh lặp `while` và các lệnh đã học

Nhiệm vụ 1. Viết chương trình in toàn bộ dãy các số tự nhiên từ 1 đến 100 trên một hàng ngang.

Hướng dẫn. Mở phần mềm Python và nhập chương trình sau:

```
k = 0
while k < 100:
    k = k + 1
    print(k, end = " ")
```

Nhiệm vụ 2. Viết chương trình in ra màn hình dãy các chữ cái tiếng Anh từ "A" đến "Z" theo ba hàng ngang trên màn hình, hai hàng ngang đầu có 10 chữ cái, hàng thứ ba có 6 chữ cái.

Hướng dẫn. Chúng ta đã biết các chữ cái tiếng Anh từ A đến Z chiếm các vị trí từ 65 đến 90 trong bảng mã ASCII. Với số thứ tự `k` của bảng mã ASCII, chúng ta sử dụng lệnh `chr(k)` trả lại kí tự tương ứng trong bảng mã này.

```
i = 0
k = 65 # bắt đầu từ số thứ tự của chữ cái A.
while k <= 90:
    i = i + 1
    if i%10 == 0:
        print(chr(k))
    else:
        print(chr(k), end = " ")
    k = k + 1
```

Với các chữ cái ở cuối hàng sẽ in ra và xuống dòng.

Với các chữ cái khác thì in ra trên một hàng ngang.



LUYỆN TẬP

1. Cho dãy số 1, 4, 7, 10,.... Viết chương trình in ra phần tử lớn nhất của dãy nhưng nhỏ hơn 100.
2. Viết chương trình đếm trong dãy 100 số tự nhiên đầu tiên có bao nhiêu số thỏa mãn điều kiện: hoặc chia hết cho 5 hoặc chia cho 3 dư 1.



VẬN DỤNG

Viết chương trình in các số tự nhiên từ 1 đến 100 ra màn hình thành 10 hàng, mỗi hàng 10 số, có dạng như sau:

```
1 2 3 ... 10
11 12 ... 20
.....
91 92 ..... 100
```

SAU BÀI NÀY EM SẼ:

- Biết được kiểu dữ liệu danh sách (list), cách khởi tạo và truy cập từng phần tử của danh sách.
- Biết và thực hiện được cách duyệt các phần tử của danh sách bằng lệnh for.
- Thực hành được một số phương thức đơn giản trên dữ liệu danh sách.



Em đã được học những kiểu dữ liệu cơ bản của Python như số nguyên, số thực, xâu kí tự, kiểu dữ liệu logic. Tuy nhiên, khi em cần lưu một dãy các số hay một danh sách học sinh thì cần kiểu dữ liệu dạng *danh sách* (còn gọi là *dãy* hay *mảng*). Kiểu dữ liệu danh sách được dùng nhiều nhất trong Python là kiểu list.

Em hãy tìm một số dữ liệu kiểu danh sách thường gặp trên thực tế.

1. KIỂU DỮ LIỆU DANH SÁCH

Hoạt động 1 Khởi tạo và tìm hiểu dữ liệu kiểu danh sách

Khởi tạo dữ liệu danh sách như thế nào? Cách truy cập, thay đổi giá trị và xoá một phần tử trong danh sách như thế nào?



Ví dụ 1. Quan sát các lệnh sau để tìm hiểu kiểu dữ liệu danh sách.

```
>>> A = [1,2,3,4,5]
>>> B = [1.5, 2, "Python", "List", 0]
>>> A[0]
1
>>> B[2]
'Python'
```

Có thể truy cập từng phần tử của danh sách thông qua chỉ số. Chỉ số của list đánh số từ 0.

Kiểu dữ liệu danh sách trong Python được khởi tạo như sau:

```
<tên list> = [<v1>, <v2>, ..., <vn>]
```

Trong đó các giá trị $\langle v_k \rangle$ có thể có kiểu dữ liệu khác nhau (số nguyên, số thực, xâu kí tự,...). Ta có thể truy cập từng phần tử của danh sách thông qua chỉ số. Chỉ số bắt đầu từ 0.

Việc chỉ số hoá từng phần tử của danh sách cho thấy, có thể dùng danh sách để biểu diễn dữ liệu tương tự như kiểu mảng trong nhiều ngôn ngữ lập trình bậc cao khác. Tuy nhiên, *danh sách của Python có một khác biệt quan trọng, đó là nó có thể gồm các phần tử có kiểu dữ liệu khác nhau.*

Ví dụ 2. Quan sát các lệnh sau để biết cách thay đổi hoặc xoá phần tử của danh sách.

```
>>> A = [1,2,3,4,5]
>>> len(A)
5
```

Lệnh `len()` tính độ dài của danh sách.

```
>>> A[1] = "One"  ← Thay đổi giá trị từng phần tử bằng lệnh gán.
>>> A
[1, 'One', 3, 4, 5]
>>> del A[4]  ← Lệnh del để xoá một phần tử của danh sách.
>>> A
[1, 'One', 3, 4]
```

Sau khi đã khởi tạo danh sách, chúng ta có thể thay đổi các giá trị của từng phần tử bằng lệnh gán hoặc xoá phần tử bằng lệnh **del**.

Ví dụ 3. Quan sát các lệnh sau để biết cách tạo danh sách rỗng (có độ dài 0) và các phép toán ghép danh sách (phép +).

```
>>> a = []  ← Lệnh tạo một danh sách rỗng có độ dài bằng 0.
>>> len(a)
0
>>> [1,2] + [3,4,5,6]  ← Các phép ghép hai danh sách.
[1, 2, 3, 4, 5, 6]
```

- List là kiểu dữ liệu danh sách (dãy, mảng) trong Python. Tạo list bằng lệnh gán với các phần tử trong cặp dấu ngoặc []. Các phần tử của danh sách có thể có các kiểu dữ liệu khác nhau. Truy cập hoặc thay đổi giá trị của từng phần tử thông qua chỉ số:
<danh sách>[<chỉ số>]
- Chỉ số của danh sách bắt đầu từ 0 đến `len() - 1`, trong đó `len()` là lệnh tính độ dài danh sách.



- Cho danh sách `A = [1, 0, "One", 9, 15, "Two", True, False]`. Hãy cho biết giá trị các phần tử:
a) `A[0]` b) `A[2]` c) `A[7]` d) `A[len(A)]`
- Giả sử `A` là một danh sách các số, mỗi lệnh sau thực hiện công việc gì?
a) `A = A + [10]` b) `del A[0]`
c) `A = [100] + A` d) `A = A[1]*25`

2. DUYỆT CÁC PHẦN TỬ CỦA DANH SÁCH

Hoạt động 2 Dùng lệnh for để duyệt danh sách

Quan sát các lệnh sau để biết cách dùng lệnh **for** duyệt lần lượt các phần tử của một danh sách.



Ví dụ 1. Duyệt và in ra từng phần tử của danh sách.

```
>>> A = [1,2,3,4,5]
>>> for i in range(len(A)):  ← Biến i chạy trên vùng chỉ số từ 0 đến len(A) - 1.
    print(A[i],end = " ")
1 2 3 4 5
```

Ví dụ 2. Duyệt và in một phần của danh sách.

```
>>> A = [3, 2, 1, 5, 6, 10, 7, 12, 18]
>>> for i in range(2,5):
    print(A[i],end = " ")
1 5 6
```

Hai ví dụ trên cho thấy dùng lệnh **for** kết hợp với lệnh **range()** để duyệt từng phần tử của danh sách.

Có thể duyệt lần lượt các phần tử của danh sách bằng lệnh **for** kết hợp với vùng giá trị của lệnh **range()**.



1. Giải thích các lệnh ở mỗi câu sau thực hiện công việc gì?

a) <pre>>>> S = 0 >>> for i in range(len(A)): if A[i] > 0: S = S + A[i] >>> print(S)</pre>	b) <pre>>>> C = 0 >>> for i in range(len(A)): if A[i] > 0: C = C + 1 >>> print(C)</pre>
--	---

2. Cho dãy các số nguyên A, viết chương trình in ra các số chẵn của A.

3. THÊM PHẦN TỬ VÀO DANH SÁCH



Python có những lệnh đặc biệt để thêm phần tử vào một danh sách. Các lệnh này được thiết kế riêng cho kiểu dữ liệu danh sách và còn được gọi là phương thức (method) của danh sách.

Hoạt động 3 Tìm hiểu lệnh thêm phần tử cho danh sách

Quan sát các lệnh sau đây để biết cách thêm phần tử vào một danh sách bằng phương thức **append()**.



Ví dụ. Thêm phần tử vào cuối danh sách.

```
>>> A = [1,2]
>>> A.append(10)
>>> A
[1, 2, 10]
```

Chú ý cách dùng phương thức **append()**: gõ tên biến danh sách, dấu ".", sau đó gõ **append**.

- Python có một số lệnh dành riêng (phương thức) cho dữ liệu kiểu danh sách. Cú pháp các lệnh đó như sau:
<danh sách>.<phương thức>
- Lệnh thêm phần tử vào cuối danh sách là <danh sách>.**append()**.



1. Sau khi thêm một phần tử vào danh sách A bằng lệnh **append()** thì độ dài danh sách A thay đổi như thế nào?

2. Danh sách A sẽ như thế nào sau các lệnh sau?

```
>>> A = [2,4,10,1,0]
>>> A.append(100)
>>> del A[1]
```



THỰC HÀNH

Khởi tạo, nhập dữ liệu, thêm phần tử cho danh sách

Nhiệm vụ 1. Nhập số n từ bàn phím, sau đó nhập danh sách n tên các bạn lớp em và in ra danh sách các tên đó, mỗi tên trên một dòng.

Hướng dẫn. Chương trình yêu cầu nhập số tự nhiên n , sau đó nhập từng tên trong danh sách, dùng phương thức `append()` để đưa dần vào danh sách.

Chú ý: Vì vùng giá trị của lệnh `range(n)` bắt đầu từ 0 nên trong thông báo nhập cần viết là `str(i+1)` để bắt đầu từ 1.

Chương trình có thể như sau:

```
dsLop = []
n = int(input("Nhập số học sinh trong lớp: "))
for i in range(n):
    name = input("Nhập họ tên học sinh thứ " + str(i+1) + ": ")
    dsLop.append(name)
print("Danh sách học sinh đã nhập:")
for i in range(len(dsLop)):
    print(dsLop[i])
```

Nhiệm vụ 2. Nhập một dãy số từ bàn phím. Tính tổng, trung bình của dãy và in dãy số trên một hàng ngang.

Hướng dẫn. Tương tự nhiệm vụ 1, chỉ khác là nhập số nguyên nên dùng lệnh `int()` để chuyển đổi dữ liệu.

```
A = []
T = 0
n = int(input("Nhập số tự nhiên n: "))
for i in range(n):
    num = int(input("Nhập số thứ " + str(i+1) + ": "))
    A.append(num)
    T = T + num
print("Dãy số đã nhập:")
for i in range(n):
    print(A[i], end = " ")
print()
print("Tổng:", T)
print("Trung bình:", T/n)
```



LUYỆN TẬP

- Viết lệnh xoá phần tử cuối cùng của danh sách A bằng lệnh `del`.
- Có thể thêm một phần tử vào đầu danh sách được không? Nếu có thì nêu cách thực hiện.



VẬN DỤNG

Cho dãy số A . Viết chương trình tìm giá trị và chỉ số của phần tử lớn nhất của A . Tương tự với bài toán tìm phần tử nhỏ nhất.

SAU BÀI NÀY EM SẼ:

- Biết cách duyệt danh sách bằng toán tử **in**.
- Biết và thực hiện được một số phương thức thường dùng với danh sách.



Trong bài trước chúng ta đã biết cách dùng lệnh `append` để thêm phần tử vào cuối một danh sách. Vậy Python có lệnh nào dùng để:

- Xoá nhanh một danh sách?
- Chèn thêm phần tử vào đầu hay giữa danh sách?
- Kiểm tra một phần tử có nằm trong một danh sách không?

1. DUYỆT DANH SÁCH VỚI TOÁN TỬ IN

Hoạt động 1 Sử dụng toán tử `in` với danh sách

Quan sát ví dụ sau để biết cách dùng toán tử `in` để duyệt một danh sách.



Ví dụ 1. Dùng toán tử `in` để kiểm tra một giá trị có nằm trong danh sách hay không.

```
>>> A = [1,2,3,4,5]
>>> 2 in A  ← Số nguyên 2 nằm trong dãy A, kết quả trả lại True.
True
>>> 10 in A ← Số 10 không nằm trong dãy A, kết quả trả lại False.
False
```

Câu lệnh dùng toán tử `in` để kiểm tra <giá trị> có trong <danh sách> không, nếu có thì trả lại `True` nếu không thì trả về `False` như sau:

```
<giá trị> in <danh sách>
```

Ví dụ 2. Sử dụng toán tử `in` để duyệt từng phần tử của danh sách.

```
>>> A = [10,11,12,13,14,15]
>>> for k in A:  ← Khi thực hiện lệnh này, biến k sẽ lần lượt nhận các
    print(k, end = " ")  giá trị từ dãy A.
10 11 12 13 14 15
```

- Toán tử `in` dùng để kiểm tra một phần tử có nằm trong danh sách đã cho không. Kết quả trả lại `True` (Đúng) hoặc `False` (Sai).
`<giá trị> in <danh sách>`
- Có thể duyệt nhanh từng phần tử của danh sách bằng toán tử `in` và lệnh `for` mà không cần sử dụng lệnh `range()`.



1. Giả sử $A = ["0", "1", "01", "10"]$. Các biểu thức sau trả về giá trị đúng hay sai?

- a) `1 in A`
- b) `"01" in A`

2. Hãy giải thích ý nghĩa từ khoá `in` trong câu lệnh sau:

```
for i in range(10):
    <các lệnh>
```

2. MỘT SỐ LỆNH LÀM VIỆC VỚI DANH SÁCH

Hoạt động 2 Tìm hiểu một số lệnh làm việc với danh sách

Quan sát ví dụ sau để tìm hiểu một số lệnh làm việc với dữ liệu kiểu danh sách.



Ví dụ 1. Lệnh `clear()` xoá toàn bộ một danh sách.

```
>>> A = [1,2,3,4,5]
>>> A.clear()  ← Sau khi thực hiện lệnh clear(), danh sách gốc trở thành rỗng.
>>> A
[]
```

Ví dụ 2. Lệnh `remove(value)` sẽ xoá phần tử đầu tiên của danh sách có giá trị `value`. Nếu không có phần tử nào như vậy thì sẽ báo lỗi.

```
>>> A = [1,2,3,4,5]
>>> A.remove(1)  ← Lệnh remove() có chức năng xoá một phần tử có giá trị cho trước.
>>> A
[2, 3, 4, 5]
```

```
>>> A.remove(10)  ← Lệnh báo lỗi nếu giá trị không có trong danh sách.
```

Traceback (most recent call last):

```
File "<pyshell#25>", line 1, in <module>
    A.remove(10)
```

ValueError: list.remove(x): x not in list

Ví dụ 3. Lệnh `insert` có chức năng chèn phần tử vào danh sách tại chỉ số cho trước.

```
>>> A = [1,2,6,10]
>>> A.insert(2,5)  ← Lệnh insert() có hai tham số cần nhập: vị trí cần chèn và giá trị được chèn. Lệnh insert(2,5) sẽ chèn số 5 tại chỉ số 2.
>>> A
[1, 2, 5, 6, 10]
```

Lệnh `insert(index,value)` sẽ chèn giá trị `value` vào danh sách tại vị trí `index` và đẩy các phần tử từ vị trí này sang phải.

Chú ý nếu `k` nằm ngoài phạm vi chỉ số của danh sách thì lệnh vẫn có tác dụng: nếu `k < 0` thì chèn vào đầu danh sách, nếu `k > len()` thì chèn vào cuối danh sách.

```
>>> A = []
>>> A.insert(-10,1)  ← Lệnh này chèn số 1 vào đầu danh sách A.
```

```
>>> A.insert(100,2) ← Lệnh này chèn số 2 vào cuối danh sách A.
```

```
>>> A
```

```
[1, 2]
```

Một số lệnh làm việc với dữ liệu danh sách:

A.append(x)	Bổ sung phần tử x vào cuối danh sách A.
A.insert(k, x)	Chèn phần tử x vào vị trí k của danh sách A.
A.clear()	Xoá toàn bộ dữ liệu của danh sách A.
A.remove(x)	Xoá phần tử x từ danh sách A.



1. Khi nào thì lệnh A.append(1) và A.insert(0,1) có tác dụng giống nhau?
2. Danh sách A trước và sau lệnh insert() là [1,4,10,0] và [1,4,10,5,0]. Lệnh đã dùng là gì?



THỰC HÀNH

Các lệnh làm việc với dữ liệu kiểu danh sách

Nhiệm vụ 1. Nhập số n từ bàn phím, sau đó nhập danh sách n tên học sinh trong lớp và in ra danh sách học sinh này, mỗi tên học sinh trên một dòng. Yêu cầu danh sách được in ra theo thứ tự ngược lại với thứ tự đã nhập.

Hướng dẫn. Chương trình sẽ yêu cầu nhập số tự nhiên n, sau đó sẽ lần lượt yêu cầu nhập n tên học sinh. Tuy nhiên do yêu cầu in danh sách học sinh theo thứ tự ngược lại so với thứ tự nhập nên cần dùng lệnh insert() để chèn tên học sinh được nhập vào đầu danh sách. Chương trình có thể như sau:

```
dsLop = []
n = int(input("Nhập số học sinh trong lớp:"))
for i in range(n):
    name = input("Nhập họ tên học sinh thứ " + str(i+1) + ": ")
    dsLop.insert(0,name)
print("Danh sách học sinh đã nhập:")
for name in dsLop:
    print(name)
```

Nhiệm vụ 2. Cho trước dãy số A. Viết chương trình xoá đi các phần tử có giá trị nhỏ hơn 0 từ A.

Hướng dẫn. Duyệt từng phần tử của dãy số A, kiểm tra nếu phần tử này nhỏ hơn 0 thì xoá đi.

Chương trình có thể như sau:

```
A = [0,1,-3,-10,5,9,-20,55]
i = 0
while i < len(A):
    if A[i] < 0:
```

```

        A.remove(A[i])
    else:
        i = i + 1
print(A)

```

Nhiệm vụ 3. Cho trước dãy số A. Viết chương trình tìm và chỉ ra vị trí đầu tiên của dãy số A mà ba số hạng liên tiếp có giá trị là 1, 2, 3. Nếu tìm thấy thì thông báo vị trí tìm thấy, nếu không thì thông báo "Không tìm thấy mẫu".

Hướng dẫn. Soạn thảo chương trình sau rồi thực hiện và kiểm tra tính đúng đắn của chương trình.

```

A = [0,4,0,1,2,3,8,9,0,1,2,3,17,-16,0,1,2]
p = [1,2,3]
pkq = -1
i = 0
while i < len(A) - 3 and pkq == -1:
    if A[i] == p[0] and A[i+1] == p[1] and A[i+2] == p[2]:
        pkq = i
    else:
        i = i + 1
if pkq >= 0:
    print("Tìm thấy mẫu ",p," tại vị trí", pkq)
else:
    print("Không tìm thấy mẫu",p)

```



LUYỆN TẬP

- Cho dãy số [1,2,2,3,4,5,5]. Viết lệnh thực hiện:
 - Chèn số 1 vào ngay sau giá trị 1 của dãy.
 - Chèn số 3 và số 4 vào danh sách để dãy có số 3 và số 4 liên nhau hai lần.
- Cho trước dãy số A. Viết chương trình thực hiện công việc sau:
 - Xoá đi một phần tử ở chính giữa dãy nếu số phần tử của dãy là số lẻ.
 - Xoá đi hai phần tử ở chính giữa của dãy nếu số phần tử của dãy là số chẵn.



VẬN DỤNG

- Viết chương trình nhập n từ bàn phím, tạo và in ra màn hình dãy số A bao gồm n số tự nhiên chẵn đầu tiên.
- Dãy số Fibonacci được xác định như sau:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \text{ (với } n \geq 2\text{)}.$$
 Viết chương trình nhập n từ bàn phím, tạo và in ra màn hình dãy số A bao gồm n số hạng đầu của dãy Fibonacci.

SAU BÀI NÀY EM SẼ:

- Hiểu được chuỗi ký tự là kiểu dữ liệu cơ bản của Python.
- Biết và thực hiện được lệnh for để xử lý chuỗi ký tự.



Em đã biết dữ liệu chuỗi ký tự (gọi tắt là chuỗi) từ Bài 16 và chúng ta có thể tạo các biến kiểu chuỗi ký tự theo nhiều cách như sau:

```
>>> s = "Thời khoá biểu"
>>> xau = 'Hoa học trò'
>>> Cau_tho = """ Mình về mình có nhớ ta
Mười lăm năm ấy thiết tha mặn nồng"""
```

Liệu có lệnh nào trích ra từng ký tự của một chuỗi ký tự? Đếm số ký tự của một chuỗi?

1. XÂU LÀ MỘT DÃY CÁC KÍ TỰ

Hoạt động 1 Tìm hiểu cấu trúc của chuỗi ký tự

Quan sát các ví dụ sau để biết cấu trúc chuỗi ký tự, so sánh với danh sách để biết sự khác nhau giữa chuỗi (string) và danh sách (list).



Ví dụ 1. Chuỗi ký tự và cách truy cập đến từng ký tự của chuỗi.

```
>>> s = "Thời khoá biểu"
>>> len(s)
14
>>> s[0]
'T'
>>> s[10]
'b'
```

Lệnh `len()` sẽ tính độ dài của chuỗi hay số lượng các ký tự có trong chuỗi.

Có thể truy cập từng ký tự của chuỗi thông qua chỉ số. Chỉ số bắt đầu từ 0.

Một chuỗi ký tự được hiểu là một dãy các ký tự. Tương tự danh sách, ta có thể truy cập từng ký tự của chuỗi thông qua chỉ số, chỉ số bắt đầu từ 0.

Ví dụ 2. Quan sát các lệnh sau để thấy sự khác nhau giữa chuỗi và danh sách.

```
>>> d = ["a", "b", "c"]
>>> d[0] = "A"
>>> s = "abc"
>>> s[0] = "A"
Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    s[0] = "A"
TypeError: 'str' object does not support item assignment
```

Python không cho phép thay đổi từng kí tự của một chuỗi. Điều này khác với danh sách.

Python không có kiểu dữ liệu kí tự. Kí tự chính là chuỗi có độ dài 1. Chuỗi rỗng được định nghĩa như sau:

```
empty = ""
```

Chuỗi kí tự trong Python là dãy các kí tự Unicode. Chuỗi có thể được coi là danh sách các kí tự nhưng không thay đổi từng kí tự của chuỗi. Truy cập từng kí tự của chuỗi qua chỉ số, chỉ số từ 0 đến độ dài `len()` - 1.



1. Các chuỗi kí tự sau có hợp lệ không?

a) "123&*()+-ABC"

b) "1010110&0101001"

c) "Tây Nguyên"

d) 11111111 = 256

2. Mỗi chuỗi hợp lệ ở Câu 1 có độ dài bằng bao nhiêu?

2. LỆNH DUYỆT KÍ TỰ CỦA CHUỖI

Hoạt động 2 Tìm hiểu lệnh duyệt từng kí tự của chuỗi

Quan sát các lệnh sau để biết cách duyệt từng kí tự của chuỗi kí tự bằng lệnh `for`. Có hai cách duyệt, theo chỉ số và theo phần tử của chuỗi kí tự.

```
>>> s = "Thời khoá biểu"
>>> for i in range(len(s)): ← Duyệt theo chỉ số với lệnh range().
    print(s[i], end = " ")
```

```
T h ờ i   k h o á   b i ể u
>>> for ch in s: ← Duyệt theo kí tự của chuỗi kí tự.
    print(ch, end = " ")
```

```
T h ờ i   k h o á   b i ể u
```



- Cách thứ nhất, biến `i` lần lượt chạy theo chỉ số của chuỗi kí tự `s`, từ 0 đến `len(s) - 1`. Kí tự tại chỉ số `i` là `s[i]`.
- Cách thứ hai duyệt theo từng kí tự của chuỗi `s`. Biến `ch` sẽ được gán lần lượt các kí tự của chuỗi `s` từ đầu đến cuối.

Chú ý: Từ khoá `in`, tùy trường hợp cụ thể, hoặc là toán tử logic dùng để kiểm tra một giá trị có mặt hay không trong một vùng giá trị/danh sách/ chuỗi, hoặc để chọn lần lượt từng phần tử trong một vùng giá trị/danh sách/ chuỗi.

```
>>> "a" in "abcd"
True
>>> "abc" in "abcd"
True
```

Có thể duyệt các kí tự của chuỗi bằng lệnh **for** tương tự như với danh sách.
 $s_1 \text{ in } s_2$ trả lại giá trị **True** nếu s_1 là chuỗi con của s_2 .



1. Sau khi thực hiện các lệnh sau, biến `skq` sẽ có giá trị bao nhiêu?

```
>>> s = "81723"
>>> skq = ""
>>> for ch in s:
    if int(ch) % 2 != 0:
        skq = skq + ch
```

2. Cho `s1 = "abc"`, `s2 = "ababcabca"`. Các biểu thức logic sau cho kết quả là đúng hay sai?

- | | |
|---------------------------------|--------------------------------|
| a) <code>s1 in s2</code> | b) <code>s1 + s1 in s2</code> |
| c) <code>"abcabca" in s2</code> | d) <code>"abc123" in s2</code> |



THỰC HÀNH

Các lệnh cơ bản làm việc với chuỗi kí tự

Nhiệm vụ 1. Viết chương trình nhập số tự nhiên n là số học sinh, sau đó nhập họ và tên học sinh. Lưu họ và tên học sinh vào một danh sách. In danh sách ra màn hình, mỗi họ tên trên một dòng.

Hướng dẫn. Chương trình có thể như sau:

```
n = int(input("Nhập số học sinh trong lớp: "))
ds_lop = []
for i in range(n):
    hoten = input("Nhập họ tên học sinh thứ " + str(i+1) + ": ")
    ds_lop.append(hoten)
print("Danh sách lớp học:")
for i in range(n):
    print(ds_lop[i])
```

Nhiệm vụ 2. Nhập một chuỗi kí tự S từ bàn phím rồi kiểm tra xem chuỗi S có chứa chuỗi con "10" không.

Hướng dẫn. Cách 1. Nếu chuỗi S chứa chuỗi con "10" thì sẽ có chỉ số k mà $S[k] = "1"$ và $S[k+1] = "0"$. Cách 2. Dùng toán tử **in** để kiểm tra chuỗi "10" có là chuỗi con của S .

Cách 1: Duyệt kí tự của chuỗi theo chỉ số.

```
S = input("Nhập chuỗi kí tự bất kì: ")
kq = False
```

```

for i in range(len(S)-1):
    if S[i] == "1" and S[i+1] == "0":
        kq = True
        break
if kq:
    print("Xâu gốc có chứa xâu '10'")
else:
    print("Xâu gốc không chứa xâu '10'")

```

Cách 2: Sử dụng toán tử **in**.

```

S = input("Nhập xâu kí tự bất kì: ")
s10 = "10"
if s10 in S:
    print("Xâu gốc có chứa xâu '10'")
else:
    print("Xâu gốc không chứa xâu '10'")

```



LUYỆN TẬP

1. Cho xâu S, viết đoạn lệnh trích ra xâu con của S bao gồm ba kí tự đầu tiên của S.
2. Viết chương trình kiểm tra xâu S có chứa chữ số không. Thông báo "S có chứa chữ số" hoặc "S không chứa chữ số nào".



VẬN DỤNG

1. Cho hai xâu s_1 , s_2 . Viết đoạn chương trình chèn xâu s_1 vào giữa s_2 , tại vị trí $\text{len}(s_2)/2$. In kết quả ra màn hình.
2. Viết chương trình nhập số học sinh và họ tên học sinh. Sau đó đếm xem trong danh sách có bao nhiêu bạn tên là "Hương".

Gợi ý: Sử dụng toán tử **in** để kiểm tra một xâu có là xâu con của một xâu khác.

SAU BÀI NÀY EM SẼ:

- Biết và thực hiện được một số lệnh thường dùng với chuỗi ký tự.



Bài toán tìm kiếm chuỗi con trong một chuỗi là một trong những bài toán tin học được ứng dụng nhiều trong thực tế. Công cụ tìm kiếm thông tin trên Internet hay lệnh tìm kiếm trong soạn thảo văn bản được xây dựng trên cơ sở bài toán tìm chuỗi con.

Cho chuỗi $c = \text{"Trường Sơn"}$ và chuỗi $m = \text{"Bước chân trên dải Trường Sơn"}$. Em hãy cho biết chuỗi c có là chuỗi con của chuỗi m không. Nếu có thì tìm vị trí của chuỗi c trong chuỗi m .

1. XÂU CON VÀ LỆNH TÌM VỊ TRÍ XÂU CON

Hoạt động 1 Một số lệnh tìm kiếm chuỗi con trong chuỗi ký tự

Quan sát các ví dụ sau để tìm hiểu cách kiểm tra chuỗi con và tìm kiếm vị trí chuỗi con trong chuỗi ký tự.



Ví dụ 1. Dùng toán tử `in` để kiểm tra một chuỗi có là chuỗi con của chuỗi khác không.

```
>>> "abc" in "123abc"
```

```
True
```

```
>>> "010" in "1101"
```

```
False
```

Biểu thức kiểm tra <chuỗi 1> nằm trong <chuỗi 2> là:

```
<chuỗi 1> in <chuỗi 2>
```

Nếu đúng thì trả lại giá trị True, nếu sai trả lại giá trị False.

Ví dụ 2. Lệnh `find()` tìm vị trí xuất hiện của một chuỗi trong chuỗi khác.

```
>>> s = "ab bc cd 123 456 00"
```

```
>>> s.find("b")
```

```
1 ← Vị trí xuất hiện đầu tiên của "b" trong chuỗi s là chỉ số 1.
```

```
>>> s.find("12")
```

```
9 ← Vị trí tìm thấy đầu tiên của "12" trong chuỗi s là chỉ số 9.
```

```
>>> s.find("AB")
```

```
-1 ← Không tìm thấy chuỗi "AB" trong chuỗi s nên trả về -1.
```

Tương tự danh sách, Python cũng có một số lệnh đặc biệt dành riêng cho chuỗi ký tự (phương thức). Cách thực hiện phương thức là:

```
<chuỗi>.<phương thức>
```

Cú pháp đơn của lệnh `find()`:

```
<chuỗi mẹ>.find(<chuỗi con>)
```

Lệnh sẽ tìm vị trí đầu tiên của chuỗi con trong chuỗi mẹ và trả về vị trí đó. Nếu không tìm thấy thì trả về -1.

Cú pháp đầy đủ của lệnh `find()`:

```
<chuỗi mẹ>.find(<chuỗi con>, start)
```

Lệnh sẽ tìm vị trí chuỗi con bắt đầu từ vị trí `start`.

Ví dụ 3

```
>>> sub = "Đà Nẵng"
>>> s = "Hà Nội - Đà Nẵng - Hồ Chí Minh"
>>> s.find(sub)
9
>>> s.find(sub, 10)
-1
```

Để tìm một chuỗi trong một chuỗi khác có thể dùng toán tử `in` hoặc lệnh `find()`. Lệnh `find()` trả về vị trí của chuỗi con trong chuỗi mẹ.



1. Biểu thức logic sau là đúng hay sai?

```
>>> "010" in "001100"
```

2. Lệnh sau trả lại giá trị gì?

```
>>> "ababababab".find("ab", 4)
```

2. MỘT SỐ LỆNH THƯỜNG DÙNG VỚI CHUỖI KÝ TỰ

Hoạt động 2 Một số lệnh thường dùng với chuỗi ký tự

Quan sát các ví dụ sau để biết cách sử dụng một số lệnh thường dùng với chuỗi ký tự như: `split()`, `join()`.



Ví dụ 1. Lệnh `split()` tách một chuỗi thành danh sách các từ.

```
>>> s = "Tiên học lễ hậu học văn"
>>> s.split()  ← Tách chuỗi dùng dấu cách để phân biệt tách.
['Tiên', 'học', 'lễ', 'hậu', 'học', 'văn']
>>> st = "0,1,2,3,4,5,6,10"
>>> st.split(",")  ← Tách chuỗi dùng dấu "," để phân biệt tách.
['0', '1', '2', '3', '4', '5', '6', '10']
```

Lệnh `split()` tách một chuỗi thành các từ và đưa vào một danh sách. Kí tự tách dùng để phân tách các từ mặc định là dấu cách, tuy nhiên có thể thay thế kí tự tách bằng kí tự khác.

Cú pháp của lệnh `split()`.

```
<chuỗi mẹ>.split(<kí tự tách>)
```

Ví dụ 2. Lệnh `join()` nối danh sách gồm các từ thành một chuỗi.

```
>>> A = ['Tiên', 'học', 'lễ', 'hậu', 'học', 'văn']
>>> " ".join(A) ← Lệnh join() này sẽ nối các phần tử
'Tiên học lễ hậu học văn' của danh sách A bởi dấu cách.

>>> B = ['0', '1', '2', '3', '4', '5', '6', '10']
>>> ", ".join(B) ← Lệnh join() này sẽ nối các phần tử
'0,1,2,3,4,5,6,10' của danh sách B bởi dấu ",".
```

Lệnh `join()` có tác dụng ngược với lệnh `split()`, có chức năng nối các phần tử (là chuỗi) của một danh sách thành một chuỗi. Cú pháp của lệnh `join()` là:

```
"kí tự nối".join(<danh sách>)
```

Python có các lệnh đặc biệt để xử lý chuỗi là `split()` dùng để tách chuỗi thành danh sách và lệnh `join()` dùng để nối danh sách các chuỗi thành một chuỗi.



Cho chuỗi kí tự: "gà,vịt,chó,lợn,ngựa,cá". Em hãy trình bày cách làm để xóa các dấu "," và thay thế bằng dấu " " trong chuỗi này.



THỰC HÀNH

Một số bài toán liên quan đến chuỗi kí tự.

Nhiệm vụ 1. Viết chương trình nhập nhiều số nguyên từ bàn phím, các số cách nhau bởi dấu cách. Khi nhập xong thông báo số lượng các số đã nhập và in các số này thành hàng ngang.

Hướng dẫn. Dữ liệu nhập vào là một chuỗi. Dùng lệnh `split()` để tách thành danh sách. Chuyển các phần tử của danh sách này thành số và in ra màn hình.

```
s = input("Nhập các số nguyên cách nhau bởi dấu cách: ")
sline = s.split()
n = len(sline)
nline = []
for x in sline:
    nline.append(int(x))
print("Bạn đã nhập", n, "số.")
for k in nline:
    print(k, end = " ")
```

Nhiệm vụ 2. Viết chương trình nhập một chuỗi kí tự có thể có nhiều dấu cách giữa các từ. Sau đó chỉnh sửa chuỗi kí tự đó sao cho giữa các từ chỉ có một dấu cách. In chuỗi kết quả ra màn hình.

Hướng dẫn. Chuyển chuỗi kí tự ban đầu thành danh sách các từ đơn bằng lệnh `split()`, sau đó nối các từ đơn này bằng lệnh `join()`.

```
s = input("Nhập đoạn văn bản: \n")
sline = s.split()
skq = " ".join(sline)
print(skq)
```

Nhiệm vụ 3. Viết chương trình nhập số tự nhiên n , rồi nhập họ tên của n học sinh. Sau đó in ra danh sách tên học sinh theo hai cột, cột 1 là tên, cột 2 là họ đệm.

Hướng dẫn. Họ tên ban đầu tách ra thành tên và họ đệm bằng lệnh `split()`. Các tên được đưa vào danh sách `ten`, các họ đệm được đưa vào danh sách `hodem`. Sau đó in ra danh sách theo yêu cầu.

```
n = int(input("Nhập số học sinh trong lớp: "))
ten = []
hodem = []
for i in range(n):
    s = input("Nhập họ tên học sinh thứ "+str(i+1)+": ")
    sline = s.split()
    m = len(sline)-1
    ten.append(sline[m])
    del sline[m]
    hodem.append(" ".join(sline))
print("Danh sách học sinh:")
for i in range(n):
    print(ten[i],hodem[i])
```



LUYỆN TẬP

- Viết chương trình nhập nhiều số (số nguyên hoặc số thực) từ bàn phím, các số cách nhau bởi dấu cách. Sau đó in ra màn hình tổng các số đã nhập.
- Viết chương trình nhập họ tên đầy đủ của người dùng, sau đó in thông báo tên và họ đệm của người đó.



VẬN DỤNG

- Viết chương trình nhập hai số tự nhiên từ bàn phím, cách nhau bởi dấu cách và đưa ra kết quả là ƯCLN của hai số này.
- Viết chương trình nhập số tự nhiên n rồi nhập n họ tên học sinh. Sau đó yêu cầu nhập một tên và thông báo số bạn có cùng tên đó trong lớp.

SAU BÀI NÀY EM SẼ:

- Biết được chương trình con là hàm.
- Biết cách tạo hàm.



Các chương trình giải những bài toán thực tế phức tạp thường có rất nhiều dòng lệnh, trong đó có không ít những khối lệnh tương ứng với một số thao tác được lặp đi lặp lại nhiều lần ở những vị trí khác nhau. Để đỡ công viết đi viết lại các khối lệnh đó, trong tổ chức chương trình viết bằng ngôn ngữ lập trình bậc cao, người ta thường gom các khối lệnh như vậy thành những chương trình con. Khi đó, trong chương trình người ta chỉ cần thay cả khối lệnh bằng một lệnh gọi chương trình con tương ứng. Trong Python, các hàm chính là các chương trình con.

Em có thể kể tên một số hàm trong số các lệnh đã học hay không? Các hàm đó có những đặc điểm chung gì?

1. MỘT SỐ HÀM THIẾT KẾ SẴN CỦA PYTHON

Hoạt động 1 Tìm hiểu một số hàm của Python

Quan sát một số câu lệnh trong Bảng 26.1 và cho biết những câu lệnh này có điểm chung gì.

Bảng 26.1. Một số lệnh trong Python

<code>abs()</code>	<code>len()</code>	<code>range()</code>	<code>bool()</code>	<code>float()</code>
<code>list()</code>	<code>round()</code>	<code>chr()</code>	<code>input()</code>	<code>ord()</code>
<code>str()</code>	<code>divmod()</code>	<code>int()</code>	<code>print()</code>	<code>type()</code>



Trước tiên, về hình thức, em có thể thấy các lệnh trên đều có các dấu mở đóng ngoặc đi sau tên lệnh. Tiếp đó, khi viết trong chương trình, bên trong các dấu ngoặc, nói chung, em có thể cần ghi thêm các tham số là các đại lượng, các biến hoặc thậm chí, trong một số trường hợp, cả các biểu thức. Ví dụ:

- Lệnh `print("Thời khoá biểu")` thực hiện việc in chuỗi kí tự "Thời khoá biểu" trong dấu ngoặc ra màn hình.
- Lệnh `x = int("52")` chuyển chuỗi "52" thành số nguyên 52.
- Lệnh `type(y)` trả lại kiểu dữ liệu của biến y.

Trong một số trường hợp bên trong dấu ngoặc có thể bỏ trống. Ví dụ, lệnh `x = input()` thực hiện yêu cầu nhập vào một chuỗi kí tự bất kì và gán cho biến x.

Các lệnh trong Bảng 26.1 chính là các chương trình con được thiết kế sẵn của Python, cho phép người dùng tùy ý sử dụng trong các chương trình của riêng mình.

Trong các ví dụ trên, dấu ngoặc bên trong ngoặc của các hàm `int()` và `print()` là **tham số của hàm**. Cú pháp câu lệnh gọi hàm trong Python có dạng chung như sau:

`<tên hàm>(<danh sách tham số hàm>)`

Python cung cấp sẵn nhiều hàm thực hiện những công việc khác nhau cho phép người dùng được tùy ý sử dụng khi viết chương trình bằng các câu lệnh gọi hàm tương ứng.



Mô tả tham số và giá trị trả lại của mỗi hàm sau: `float()`, `str()`, `len()`, `list()`.

2. THIẾT LẬP CÁC HÀM TỰ ĐỊNH NGHĨA



Ngoài các hàm thiết kế sẵn, Python còn cho phép người dùng tự thiết lập các hàm của riêng mình (còn gọi là các hàm tự định nghĩa).

Hoạt động 2 Cách thiết lập hàm trong Python

Quan sát các ví dụ sau để biết cách viết hàm.



Ví dụ 1. Cách viết hàm có trả lại giá trị.

```
>>> def inc(n):  
    return n+1
```

```
>>> inc(3)  
4
```

Tên hàm: `inc`.
Tham số hàm: số `n`.
Giá trị trả lại: số `n + 1`.

Ví dụ 2. Cách viết hàm không trả lại giá trị.

```
>>> def thong_bao(msg):  
    print("Xin chào bạn", msg)  
    return
```

```
>>> thong_bao("Trần Quang Minh")  
Xin chào bạn Trần Quang Minh
```

Tên hàm: `thong_bao`.
Tham số hàm: chuỗi ký tự `msg`.
Giá trị trả lại: không có.

Hàm trong Python được định nghĩa bằng từ khóa `def`, theo sau là tên hàm (tên hàm sẽ theo quy tắc đặt tên định danh). Hàm có thể có hoặc không có tham số. Khối lệnh mô tả hàm được viết sau dấu `:` và viết lùi vào, thẳng hàng. Hàm có thể có hoặc không có giá trị trả lại sau từ khóa `return`.

Cú pháp thiết lập hàm có trả lại giá trị.

```
def <tên hàm>(<tham số>):  
    <khối lệnh>  
    return <giá trị>
```

Cần có lệnh **return** <giá trị>. Hàm sẽ kết thúc khi gặp lệnh **return** và trả lại <giá trị>.

Cú pháp thiết lập hàm không trả lại giá trị.

```
def <tên hàm>(<tham số>):  
    <khối lệnh>  
    return
```

Lệnh **return** không có giá trị trả lại. Hàm kết thúc khi gặp lệnh **return**. Nếu hàm không trả lại giá trị thì có thể không cần lệnh **return**.

Để thiết lập hàm trả lại giá trị, câu lệnh **return** trong khai báo hàm cần có <giá trị> đi kèm. Để thiết lập hàm không trả lại giá trị có thể dùng lệnh **return** không có <giá trị> hoặc không cần có **return**.



Quan sát các hàm sau, giải thích cách thiết lập và chức năng của mỗi hàm.

a)

```
def Nhap_xau():  
    msg = input("Nhập một xâu: ")  
    return msg
```

b)

```
def Inday(n):  
    for k in range(n):  
        print(k, end = " ")
```



THỰC HÀNH

Thiết lập hàm trong Python.

Nhiệm vụ 1. Viết hàm yêu cầu người dùng nhập họ tên rồi đưa lời chào ra màn hình.

Hướng dẫn. Chương trình có thể như sau:

```
def meeting():  
    ten = input("Nhập họ tên của em: ")  
    print("Xin chào ",ten)  
meeting()
```

Nhiệm vụ 2. Viết hàm **prime(n)** với tham số là số tự nhiên n và trả lại **True** nếu n là số nguyên tố, trả lại **False** nếu n không phải là số nguyên tố.

Hướng dẫn. Số nguyên tố là số tự nhiên lớn hơn 1, không có ước nào ngoài 1 và chính nó. Để thiết lập hàm **prime(n)** chúng ta cần tính số ước thực sự của n (từ 1 đến $n - 1$). Biến C dùng để đếm số các ước thực sự của n . Khi đó, n sẽ là số nguyên tố khi và chỉ khi $C = 1$.

Hàm **prime(n)** và chương trình có thể được thiết lập như sau:

```
def prime(n):  
    C = 0  
    k = 1
```

Ban đầu đặt $k = 1$. Vòng lặp sẽ tăng k lên 1 đơn vị cho đến khi $k = n$ thì dừng. Với mỗi k , kiểm tra nếu k là ước của n thì tăng C lên 1.

```

while k < n:
    if n%k == 0:
        C = C + 1
    k = k + 1
if C == 1:
    return True
else:
    return False

```



LUYỆN TẬP

- Viết hàm với tham số là số tự nhiên n in ra các số là ước nguyên tố của n .
Gợi ý: Sử dụng hàm `prime()` trong phần thực hành.
- Viết hàm `numbers(s)` đếm số các chữ số có trong chuỗi s .
Ví dụ `numbers("0101abc") = 4`.



VẬN DỤNG

- Trong khi viết hàm có thể có nhiều lệnh `return`. Quan sát hàm sau và giải thích ý nghĩa của những lệnh `return`. Hàm này có điểm gì khác so với hàm `prime()` đã được mô tả trong phần thực hành.

```

def prime(n):
    if n < 2:
        return False
    C = 0
    k = 2
    while k < n:
        if n%k == 0:
            return False
        k = k + 1
    return True

```
- Viết chương trình yêu cầu nhập từ bàn phím một chuỗi kí tự, sau đó thông báo:
 - Tổng số các kí tự là chữ số của chuỗi.
 - Tổng số các kí tự là chữ cái tiếng Anh trong chuỗi.
Viết hàm cho mỗi yêu cầu trên.

SAU BÀI NÀY EM SẼ:

- Biết cách thiết lập các tham số của hàm. Hiểu được cách truyền giá trị thông qua đối số hàm.
- Biết viết chương trình có sử dụng chương trình con.



Quan sát các lệnh sau và cho biết sự khác nhau giữa tham số (parameter) và đối số (argument).

Tham số

```
>>> def f_sum(a,b,c):
        return a+b+c
```

Đối số

```
>>> x,y =5,3
>>> f_sum(10,x,y):
18
```

1. THAM SỐ VÀ ĐỐI SỐ CỦA HÀM

Hoạt động 1 Phân biệt tham số và đối số

Quan sát ví dụ sau, tìm hiểu cách dữ liệu được truyền qua tham số vào hàm. Thảo luận để giải thích kết quả.



Ví dụ. Cách truyền dữ liệu qua tham số.

```
1 >>> def f(a,b,c):
2     return a+b+c
3 >>> f(1,2,3)
4 6
5 >>> x,y,z = 10,20,5
6 >>> f(x,y,z)
7 35
8 >>> f(a,b,c)
9 Traceback (most recent call last):
10   File "<pyshell#6>", line 1, in <module>
11     f(a,b,c)
12 NameError: name 'a' is not defined
13 >>>
```

Hàm f() có ba tham số a, b, c.

Hàm f() được gọi với ba giá trị cụ thể.

Hàm f() được gọi với ba biến đã có giá trị.

Lời gọi hàm bị lỗi nếu các tham số được truyền vào chưa có giá trị.

Giải thích:

- Dòng 1: Hàm $f()$ đã được định nghĩa với ba tham số a, b, c . Hàm có trả lại giá trị là $a + b + c$. Chú ý trong định nghĩa hàm, các tham số được coi như biến.
- Dòng 3: Hàm $f()$ được gọi với ba giá trị cụ thể là 1, 2, 3. Các giá trị được truyền qua tham số được gọi là đối số. Đối số tại dòng 3 là các số cụ thể.
- Dòng 6: Hàm $f()$ được gọi với ba biến x, y, z , đã được gán giá trị (dòng 5). Các biến được truyền qua tham số được gọi là đối số, kết quả trả lại là $x + y + z$.
- Dòng 10: Hàm $f()$ được gọi với ba biến a, b, c không xác định giá trị nên lời gọi hàm $f(a, b, c)$ báo lỗi do không xác định được giá trị của a, b, c .

Tham số của hàm được định nghĩa khi khai báo hàm và được dùng như biến trong định nghĩa hàm. **Đối số** là giá trị được truyền vào khi gọi hàm. Khi gọi hàm, các **tham số (parameter)** sẽ được truyền bằng giá trị thông qua **đối số (argument)** của hàm, số lượng giá trị được truyền vào hàm bằng với số tham số trong khai báo của hàm.



1. Một hàm khi khai báo có một tham số, nhưng khi gọi hàm có thể có hai đối số được không?
2. Giả sử hàm f có hai tham số x, y khi khai báo, hàm sẽ trả lại giá trị $x + 2y$. Lời gọi hàm $f(10, a)$ có lỗi hay không?

2. CÁCH SỬ DỤNG CHƯƠNG TRÌNH CON

Hoạt động 2 Khi nào nên sử dụng chương trình con?

Bài toán đưa ra là viết chương trình chính yêu cầu nhập số tự nhiên n từ bàn phím và in các số nguyên tố nhỏ hơn hoặc bằng n ra màn hình. Trong phần thực hành của Bài 26 em đã biết hàm $\text{prime}(n)$ kiểm tra số n có là số nguyên tố.

Em sẽ viết chương trình giải bài toán này như thế nào?



Ví dụ 1. Việc kiểm tra một số có là số nguyên tố được lặp đi lặp lại từ 1 đến n và do đó nên sử dụng hàm $\text{prime}(n)$ để kiểm tra sẽ giúp chương trình cấu trúc rõ ràng và dễ hiểu hơn.

Chương trình hoàn chỉnh giải bài toán trên có thể được viết như sau:

```
def prime(n):  
    C = 0  
    k = 1  
    while k < n:  
        if n%k == 0:  
            C = C + 1  
        k = k + 1
```

```

if C == 1:
    return True
else:
    return False
# Chương trình chính
n = int(input("Nhập số tự nhiên n: "))
for k in range(1, n+1):
    if prime(k):
        print(k, end = " ")

```

Sử dụng hàm prime đã định nghĩa ở trên để kiểm tra số k: Hàm prime(k) sẽ trả lại True nếu k là số nguyên tố.

Ví dụ 2. Chương trình sử dụng chương trình con.

Cho trước hai dãy số B, C, chương trình chính cần tính tổng các số hạng dương của mỗi dãy này. Chúng ta sẽ thiết lập hàm tongduong(A) để tính tổng các số hạng lớn hơn 0 của một dãy A. Chương trình chính sẽ gọi hàm tongduong(A).

Chương trình có thể như sau:

```

>>> def tongduong(A):
    S = 0
    for k in A:
        if k > 0:
            S = S + k
    return S
# Chương trình chính
>>> A = [0, 2, -1, 5, 10, -3]
>>> B = [1, -10, -11, 8, 2, 0, -5]
>>> tongduong(A)
17
>>> tongduong(B)
11
>>> print(B)

```

Sử dụng hàm tongduong tính tổng các số dương của dãy A

Sử dụng hàm tongduong tính tổng các số dương của dãy B

Sử dụng chương trình con có thể giúp phân chia việc giải một bài toán lớn thành giải quyết các bài toán nhỏ và phát huy được tinh thần làm việc nhóm; Chương trình chính có cấu trúc rõ ràng, dễ hiểu hơn; Nếu cần hiệu chỉnh, phát triển và nâng cấp cũng thuận tiện hơn.



1. Sử dụng hàm prime, em hãy viết chương trình in ra các số nguyên tố trong khoảng từ m đến n, với m, n là hai số tự nhiên và $1 < m < n$.
2. Em hãy nêu một công việc/bài toán nào đó mà có thể sử dụng hàm để giải.



THỰC HÀNH

Truyền giá trị cho đối số của hàm.

Nhiệm vụ 1. Thiết lập hàm `f_sum(A, b)` có chức năng tính tổng các số của danh sách A theo quy định sau:

- Nếu `b = 0` thì tính tổng các số của danh sách A.
- Nếu `b` khác 0 thì chỉ tính tổng các số dương của A.

Hướng dẫn. Chương trình luôn kiểm tra giá trị của đối số `b` khi tính tổng các số của danh sách A.

Chương trình có thể như sau:

```
def f_sum(A, b):
    S = 0
    for x in A:
        if b == 0:
            S = S + x
        else:
            if x > 0:
                S = S + x
    return S
```

Nhiệm vụ 2. Thiết lập hàm `f_dem(msg, sep)` có chức năng đếm số các từ của một chuỗi `msg` với kí tự tách từ là `sep`.

Ví dụ:

```
f_dem("Mùa thu lịch sử", " ") ← Trả lại giá trị 4.
f_dem("Mùa thu lịch sử", "-") ← Trả lại giá trị 1.
```

Hướng dẫn. Để tách chuỗi `msg` thành các từ, chúng ta dùng lệnh `split()`. Tham số `sep` chính là tham số của lệnh `split()`.

Chương trình có thể như sau:

```
def f_dem(msg, sep):
    xlist = msg.split(sep)
    return len(xlist)
```

Nhiệm vụ 3. Thiết lập hàm `merge_str(s1, s2)` với `s1, s2` là hai chuỗi cần gộp.

Hàm này sẽ gộp hai chuỗi `s1, s2` theo cách, lấy lần lượt từng kí tự của `s1, s2` đưa vào chuỗi kết quả. Nếu có một chuỗi hết kí tự thì đưa phần còn lại của chuỗi dài hơn vào chuỗi kết quả. Ví dụ nếu `s1 = "1111"`, `s2 = "0000"`, `c = 1` thì chuỗi kết quả là `"10101010"`.

Hướng dẫn. Gọi S là xâu kết quả sau khi gộp hai xâu s1 và s2, chương trình có thể như sau:

```
def merge_str(s1, s2):  
    S = ""  
    l1 = len(s1)  
    l2 = len(s2)  
    l = min(l1, l2)  
    for i in range(l):  
        S = S + s1[i] + s2[i]  
    if l1 < l2:  
        for i in range(l, l2):  
            S = S + s2[i]  
    if l2 < l1:  
        for i in range(l, l1):  
            S = S + s1[i]  
    return S
```



LUYỆN TẬP

1. Thiết lập hàm power(a, b, c) với a, b, c là số nguyên. Hàm trả lại giá trị $(a+b)^c$.
2. Viết chương trình thực hiện: Nhập hai số tự nhiên từ bàn phím, hai số cách nhau bởi dấu cách. Tính và in ra tổng của các số này.

Yêu cầu sử dụng hàm khi viết chương trình.



VẬN DỤNG

1. Viết chương trình thực hiện: Nhập hai số tự nhiên từ bàn phím, hai số cách nhau bởi dấu phẩy, in ra ước chung lớn nhất (UCLN) của hai số.

Yêu cầu sử dụng hàm khi viết chương trình.

2. Thiết lập hàm change() có hai tham số là xâu ho_ten và số c. Hàm sẽ trả lại xâu kí tự ho_ten là chữ in hoa nếu c = 0. Nếu tham số c khác 0 thì hàm trả lại xâu ho_ten là chữ in thường.

Gợi ý: Sử dụng các phương thức s.upper() và s.lower() để chuyển đổi các kí tự của xâu s sang chữ in hoa và in thường.

SAU BÀI NÀY EM SẼ:

- Biết và trình bày được ý nghĩa của phạm vi hoạt động của biến trong chương trình và hàm.



1. Một biến được định nghĩa trong chương trình chính (bên ngoài các hàm) thì sẽ được sử dụng như thế nào bên trong các hàm?
2. Một biến được khai báo bên trong một hàm thì có sử dụng được ở bên ngoài hàm đó hay không?

Bài này sẽ giúp em tìm câu trả lời cho các câu hỏi trên.

1. PHẠM VI CỦA BIẾN KHAI BÁO TRONG HÀM

Hoạt động 1 Phạm vi của biến khi khai báo trong hàm

Quan sát các lệnh sau để tìm hiểu phạm vi có hiệu lực của biến khi khai báo bên trong một hàm.



Các biến được khai báo bên trong một hàm chỉ được sử dụng bên trong hàm đó. Chương trình chính không sử dụng được.

```
>>> def func(a,b):
    n = 10
    a = a*2
    b = a+b
    return a+b+n
```

Bên trong hàm này có các biến n, a, b đang hoạt động. n = 10, a và b được thay đổi.

```
>>> a = 1
>>> b = 2
>>> func(a,b)
16
>>> a,b
(1, 2)
>>> n
```

Đây là các biến bên ngoài hàm: a, b. Các biến này được gán: a = 1, b = 2.

Sau khi chạy hàm, các biến a, b vẫn không thay đổi.

```
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    n
NameError: name 'n' is not defined
```

Biến n chỉ có tác dụng bên trong hàm func, gọi bên ngoài hàm này sẽ bị báo lỗi.

Trong Python tất cả các biến khai báo bên trong hàm đều có tính địa phương (cục bộ), không có hiệu lực ở bên ngoài hàm.



1. Giả sử có các lệnh sau:

```
>>> a,b = 1,2
>>> def f(a,b):
    a = a + b
    b = b*a
    return a + b
```

Giá trị của a, b bằng bao nhiêu sau khi thực hiện lệnh sau?

a) $f(1, 2)$ b) $f(10, 20)$

2. Ta có thể khai báo một biến bên trong hàm trùng tên với biến đã khai báo trước đó bên ngoài hàm không?

2. PHẠM VI CỦA BIÊN KHAI BẢO NGOÀI HÀM

Hoạt động 2 Phạm vi của biến khi khai báo bên ngoài hàm

Quan sát các lệnh sau, tìm hiểu phạm vi có hiệu lực của biến khi khai báo bên ngoài một hàm.



Ví dụ 1. Biểu khai báo bên ngoài hàm không có tác dụng bên trong hàm.

```
>>> def f(n):
    t = n + 1
    return t
```

```
>>> t = 10
```

```
>>> f(5)
```

6

```
>>> t
```

10

Trong chương trình chính biến t được khai báo bên ngoài hàm $f()$ và gán giá trị 10. Khi gọi $f(5)$, t sẽ được gán 6. Hàm trả lại giá trị 6. Nhưng khi thoát khỏi $f()$, t vẫn có giá trị 10. Do vậy biến t không có tác dụng bên trong hàm $f()$.

Ví dụ 2. Bên trong hàm có thể truy cập để sử dụng giá trị của biến đã khai báo trước đó ở bên ngoài hàm.

```
>>> def f(a,b):
    return a + b + N
```

```
>>> N = 10
```

```
>>> f(1, 2)
```

13

Trong chương trình chính, biến N được khai báo và gán giá trị 10. Khi gọi hàm $f(1,2)$, giá trị trả lại là biểu thức có N tham gia. Vậy trong hàm $f()$ được phép truy cập giá trị của biến N.

Lưu ý: Nếu muốn biến bên ngoài vẫn có tác dụng bên trong hàm thì cần khai báo lại biến này bên trong hàm với từ khoá **global**.

```
>>> def f(n):  
    global t  ← Biến t được khai báo với từ khoá global.  
    t = 2*n + 1  
    return t  
    ← Biến t khai báo trong chương trình chính và được  
    ← gán giá trị 10.  
>>> t = 10  
>>> f(1)  ← Sau khi thực hiện f(1), giá trị t đã thay đổi.  
3  
>>> t  
3
```

Biến đã khai báo bên ngoài sẽ không có tác dụng bên trong hàm như một biến. Nếu muốn có tác dụng thì cần khai báo lại biến này trong hàm với từ khoá **global**.



Giải sử hàm $f(x,y)$ được định nghĩa như sau:

```
>>> def f(x,y):  
    a = 2*(x + y)  
    print(a + n)
```

Kết quả nào được in ra khi thực hiện các lệnh sau?

```
n = 10  
f(1,2)
```



THỰC HÀNH

Phạm vi của biến

Nhiệm vụ 1. Viết hàm với đầu vào là danh sách A chứa các số và số thực x. Hàm trả lại một danh sách kết quả B từ danh sách A bằng cách chỉ giữ lại các phần tử lớn hơn hoặc bằng x.

Hướng dẫn. Biến B kiểu danh sách cần được định nghĩa trong hàm và được bổ sung thêm các phần tử từ A nếu thoả mãn điều kiện lớn hơn hoặc bằng x.

```
def Select(A,x):  
    B = []  
    for k in range(len(A)):  
        if A[k] >= x:  
            B.append(A[k])  
    return B
```

Nhiệm vụ 2. Viết hàm với đầu vào là chuỗi ký tự Str và số c, đầu ra là danh sách các từ được tách ra từ chuỗi Str nhưng đã được chuyển thành chữ in hoa hoặc chữ in thường, hoặc chỉ chuyển ký tự đầu các từ thành chữ in hoa tùy thuộc vào tham số đầu vào c như sau:

- Nếu $c = 0$, danh sách B là các từ được chuyển thành chữ in hoa.
- Nếu $c = 1$, danh sách B là các từ được chuyển thành chữ in thường.
- Nếu $c = 2$, danh sách B là các từ được chuyển viết chữ hoa kí tự đầu của mỗi từ.

Hướng dẫn. Chúng ta cần sử dụng các lệnh sau:

`Str.upper()` – chuyển kí tự của xâu thành chữ in hoa.

`Str.lower()` – chuyển kí tự của xâu thành chữ in thường.

`Str.title()` – chuyển kí tự đầu mỗi từ của xâu thành chữ in hoa, các kí tự khác chuyển về chữ thường.

Hàm được định nghĩa có dạng **Tach_tu(Str, c)**. Đầu tiên xâu Str cần được tách từ bằng lệnh `split()`. Sau đó danh sách kết quả sẽ được chuyển đổi chữ in hoa, in thường sử dụng một trong các lệnh trên tùy thuộc vào giá trị của đối số c.

```
def Tach_tu(Str, c):
    A = Str.split()
    for k in range(len(A)):
        if c == 0:
            A[k] = A[k].upper()
        if c == 1:
            A[k] = A[k].lower()
        if c == 2:
            A[k] = A[k].title()
    return A
```

Nhiệm vụ 3. Viết chương trình yêu cầu thực hiện lần lượt các việc sau, mỗi việc cần được thực hiện bởi một hàm:

1. Nhập từ bàn phím một dãy các số nguyên, mỗi số cách nhau bởi dấu cách. Chuyển các số này vào danh sách A và in danh sách A ra màn hình.
2. Trích từ danh sách A ra một danh sách B gồm các phần tử lớn hơn 0. In danh sách B ra màn hình.
3. Trích từ danh sách A ra một danh sách C gồm các phần tử nhỏ hơn 0. In danh sách C ra màn hình.

Hướng dẫn. Với mỗi việc trên được viết thành một hàm. Toàn bộ chương trình có thể như sau:

```
def Nhap_Dulieu():
    s = input("Nhập các số nguyên cách nhau bởi dấu cách: ")
    A = s.split()
    for k in range(len(A)):
        A[k] = int(A[k])
    return A
def getB(A):
    B = []
```

```

    for x in A:
        if x > 0:
            B.append(x)
    return B
def getC(A):
    C = []
    for x in A:
        if x < 0:
            C.append(x)
    return C
# Chương trình chính
A = Nhap_Dulieu()
print("Danh sách A:",A)
B = getB(A)
C = getC(A)
print("Danh sách B:",B)
print("Danh sách C:",C)

```



LUYỆN TẬP

1. Viết hàm với đầu vào, đầu ra như sau:

- Đầu vào là danh sách sList, các phần tử là xâu kí tự.
- Đầu ra là danh sách cList, các phần tử là kí tự đầu tiên của các xâu kí tự tương ứng trong danh sách sList.

2. Viết hàm Tach_day() với đầu vào là danh sách A, đầu ra là hai danh sách B, C được mô tả như sau:

- Danh sách B thu được từ A bằng cách lấy ra các phần tử có chỉ số chẵn.
- Danh sách C thu được từ A bằng cách lấy ra các phần tử có chỉ số lẻ.



VẬN DỤNG

1. Viết hàm có hai tham số đầu vào là m, n. Đầu ra trả lại hai giá trị là:

- ƯCLN của m, n.
- Bội chung nhỏ nhất (BCNN) của m, n.

Gợi ý: Sử dụng công thức $ƯCLN(m, n) \times BCNN(m, n) = m \times n$.

2. Viết chương trình nhập ba số tự nhiên từ bàn phím day, month, year, các số cách nhau bởi dấu cách. Các số này biểu diễn giá trị của ngày, tháng, năm nào đó. Chương trình cần kiểm tra và in ra thông báo số liệu đã nhập vào đó có hợp lệ hay không.

SAU BÀI NÀY EM SẼ:

- Biết và phân loại được một số loại lỗi chương trình.
- Biết được một vài lỗi ngoại lệ thường gặp.



Một chương trình hoàn chỉnh được mô tả như Hình 29.1: Tiếp nhận các dữ liệu đầu vào, xử lý theo yêu cầu bài toán và đưa ra kết quả đúng theo yêu cầu. Theo em nếu chương trình có lỗi, thì các lỗi này sẽ như thế nào và có thể ở đâu?



Hình 29.1

1. NHẬN BIẾT LỖI CHƯƠNG TRÌNH

Hoạt động 1 Nhận biết và phân biệt một số loại lỗi chương trình

Quan sát các trường hợp chương trình gặp lỗi như sau, từ đó nhận biết và phân biệt một số loại lỗi của chương trình.



Trường hợp 1. Người lập trình viết sai cú pháp lệnh, chương trình lập tức dừng lại và thông báo lỗi cú pháp.

```
>>> while True print("Hello")
```

```
SyntaxError: invalid syntax
```

Trường hợp 2. Người dùng nhập dữ liệu sai, chương trình thông báo lỗi nhập dữ liệu không đúng khuôn dạng.

```
>>> n = int(input("Nhập số nguyên n: "))
```

```
Nhập số nguyên n: 1.5
```

```
Traceback (most recent call last):
```

```
File "<pyshell#0>", line 1, in <module>
```

```
n = int(input("Nhập số nguyên n: "))
```

```
ValueError: invalid literal for int() with base 10: '1.5'
```

Trường hợp 3. Chương trình thông báo lỗi chỉ số vượt quá giới hạn cho phép.

```
Array_b1.py
```

```
A = [1,3,10,0]
```

```
for i in range(5):
```

```
    print(A[i],end = " ")
```

Khi chạy chương trình sẽ báo lỗi:

```
1 3 10 0 Traceback (most recent call last):  
  File "C:\Python\Array_b1.py", line 3, in <module>  
    print(A[i],end = " ")
```

IndexError: list index out of range

Trường hợp 4. Chương trình thực hiện bình thường nhưng kết quả không đúng với yêu cầu của bài toán. Đây là lỗi logic bên trong chương trình.

#Tính tổng của ba số nguyên dương đầu tiên

```
>>> s = 0  
>>> for i in range(3)  
    s = s + i  
>>> print(s)  
3
```

Chương trình cho kết quả là 3 mà kết quả đúng là $1 + 2 + 3 = 6$. Lí do là hàm `range(3)` trả lại vùng giá trị là 0, 1, 2 chứ không phải là 1, 2, 3.

Giải thích:

- Với trường hợp 1, chương trình dừng và thông báo lỗi **Syntax Error** (lỗi cú pháp).
- Với trường hợp 2, khi người dùng nhập dữ liệu sai, hàm `int()` không thể thực hiện được, chương trình dừng lại và báo lỗi. Mã lỗi là **ValueError**. Đây là lỗi **Runtime** (lỗi trong khi đang thực hiện) hay còn gọi là lỗi ngoại lệ (Exceptions error).
- Với trường hợp 3, chương trình phát hiện lỗi chỉ số vượt quá giới hạn tại dòng 3. Chương trình dừng và báo lỗi. Mã lỗi là **IndexError**. Đây là lỗi **Runtime**.
- Với trường hợp 4, chương trình không còn lỗi **Runtime**, nhưng kết quả đưa ra sai. Không có mã lỗi nào được trả lại. Đây là lỗi ngữ nghĩa hoặc lỗi logic bên trong chương trình.

- Tổng thể có thể phân biệt lỗi chương trình Python làm ba loại:
 1. Lỗi khi có lệnh viết sai cú pháp hoặc sai cấu trúc ngôn ngữ Python quy định. Chương trình sẽ lập tức dừng và thông báo lỗi **Syntax Error**.
 2. Lỗi khi không thể thực hiện một lệnh trong chương trình. Chương trình dừng lại và thông báo một mã lỗi. Lỗi này gọi là lỗi ngoại lệ (**Exceptions Error**), mã lỗi trả lại gọi là mã lỗi ngoại lệ.
 3. Chương trình chạy không lỗi ngoại lệ, nhưng kết quả đưa ra sai, không chính xác. Đây là lỗi logic bên trong chương trình.
- Với mỗi loại lỗi trên, cách xử lý và kiểm soát lỗi sẽ khác nhau.



1. Khi gõ sai cú pháp một lệnh, chương trình sẽ dừng lại và báo lỗi, đó là lỗi loại gì?
2. Bài toán yêu cầu sắp xếp dãy số ban đầu thành dãy tăng dần. Giả sử dãy số ban đầu là [3, 1, 8, 10, 0]. Kết quả thu được dãy [1, 3, 8, 10, 0]. Chương trình có lỗi không? Nếu có thì lỗi đó thuộc loại gì?

2. MỘT SỐ LỖI NGOẠI LỆ THƯỜNG GẶP



Chúng ta đã biết, nếu gặp lỗi ngoại lệ, chương trình Python sẽ dừng lại và báo lỗi. Một trong những vấn đề được đưa ra khi kiểm soát lỗi là làm thế nào để vẫn phát hiện lỗi, xử lý lỗi nhưng chương trình không bị dừng lại trong khi thực hiện.

Hoạt động 2 Nhận biết một số lỗi ngoại lệ thường gặp

Đọc, thảo luận để nhận biết một số lỗi ngoại lệ thường gặp trong chương trình Python.



Mã lỗi ngoại lệ	Mô tả lỗi
ZeroDivisionError	Lỗi này xảy ra khi lệnh thực hiện phép chia cho giá trị 0.
IndexError	Lỗi xảy ra khi lệnh cố gắng truy cập phần tử của danh sách nhưng chỉ số vượt quá giới hạn.
NameError	Lỗi xảy ra khi chương trình muốn tìm một tên nhưng không thấy. Ví dụ khi lệnh gọi một hàm nhưng không có hàm đó.
TypeError	Lỗi kiểu dữ liệu. Một số ví dụ lỗi loại này: – Lệnh truy cập một phần tử của danh sách nhưng chỉ số không là số nguyên. – Lệnh tính biểu thức số nhưng lại có một toán hạng không phải là số.
ValueError	Lỗi liên quan đến giá trị của đối tượng. Lỗi khi thực hiện lệnh chuyển đổi kiểu dữ liệu, đối số của hàm có giá trị mà hàm không hỗ trợ. Ví dụ khi thực hiện lệnh <code>int("1.55")</code> sẽ sinh lỗi loại này.
IndentationError	Lỗi khi các dòng lệnh thụt vào không thẳng hàng hoặc không đúng vị trí.
SyntaxError	Lỗi cú pháp.



Hãy nêu mã lỗi ngoại lệ của mỗi lệnh sau nếu xảy ra lỗi.

- a) `A[1.5]` b) `int("abc")`
c) `"10"*3.5` d) `12 + x(10)`



THỰC HÀNH

Lập trình và kiểm tra khả năng sinh lỗi khi chạy chương trình.

Nhiệm vụ 1. Viết chương trình nhập các số nguyên m, n từ bàn phím, cách nhau bởi dấu cách. Chương trình đưa ra tổng, hiệu, thương của hai số đã nhập.

Hướng dẫn. Chương trình chính là khối các lệnh nhập từ bàn phím hai số nguyên m, n. Các số này được nhập bằng lệnh `input()`, kết quả là một chuỗi ký tự. Chuỗi này sẽ được tách thành danh sách các chuỗi con bằng lệnh `split()`. Kết quả thu được sẽ

chuyển đổi thành hai số m, n bằng lệnh `int()`. Nhập chương trình sau và kiểm tra khả năng sinh lỗi khi chạy chương trình.

```
s = input("Nhập hai số m, n cách nhau bởi dấu cách: ")
sline = s.split()
m, n = int(sline[0]),int(sline[1])
print("Tổng, hiệu, thương 2 số đã nhập là:",m+n, m-n,m/n)
```

Gợi ý. Các khả năng sinh lỗi của chương trình:

- Các số m, n khi nhập vào không là số nguyên.
- Giữa hai số m, n không có dấu cách.
- Số n nhập vào là số 0.

Nhiệm vụ 2. Viết chương trình nhập số tự nhiên n và nhập lần lượt n số nguyên đưa vào danh sách số A. Sau khi nhập xong in danh sách A ra màn hình.

Hướng dẫn. Nhập chương trình sau và kiểm tra khả năng sinh lỗi khi chạy.

```
n = int(input("Nhập số tự nhiên n: "))
A = []
for k in range(n):
    num = int(input("Nhập số thứ "+str(k+1)+" : "))
    A.append(num)
print("Dãy đã nhập:",A)
```

Gợi ý. Các khả năng sinh lỗi của chương trình:

- Số n được nhập không là số nguyên.
- Mỗi số hạng của danh sách nhập vào không là số nguyên.



LUYỆN TẬP

1. Các lệnh sau có sinh lỗi chương trình không? Nếu có thì mã lỗi là gì?

<pre>>>> A = [1, 3, 5, 10, 0] >>> for k in range(1, len(A)+1): print(A[k])</pre>	<pre>>>> s1, s2 = "101010", 1010101 >>> s = s1 + s2</pre>
--	---

2. Để tính giá trị trung bình của một danh sách số A, người lập trình đã dùng lệnh sau để tính:

```
gttb = sum(A)/len(A)
```

lệnh này có thể sinh lỗi ngoại lệ không? Nếu có thì là những lỗi gì?



VẬN DỤNG

- Giả sử em được yêu cầu viết một chương trình nhập số tự nhiên n từ bàn phím, kết quả đưa ra là danh sách các ước số thực sự của n, tính cả 1 và không tính n. Hãy viết chương trình và kiểm tra các khả năng sinh lỗi khi thực hiện chương trình.
- Em hãy viết một chương trình nhỏ để khi chạy sẽ sinh mã lỗi `NameError`.

SAU BÀI NÀY EM SẼ:

- Biết được một vài phương pháp đơn giản kiểm thử chương trình.
- Biết được một vài cách gỡ lỗi đơn giản một chương trình.



Bài học trước em đã biết khái niệm lỗi ngoại lệ khi chạy chương trình Python. Tuy nhiên, một chương trình chạy không có lỗi ngoại lệ (chương trình không bị dừng) thì không có nghĩa là chương trình không có lỗi. Thậm chí các "lỗi" không tường minh này (các lỗi này được gọi bug) càng khó phát hiện và khó sửa.

Theo em, làm thế nào để kiểm tra (test) và gỡ lỗi (debug) một chương trình? Môi trường lập trình có công cụ nào hỗ trợ việc đó không?

1. MỘT VÀI PHƯƠNG PHÁP KIỂM THỬ CHƯƠNG TRÌNH

Hoạt động

Tìm hiểu một số phương pháp kiểm thử chương trình

Đọc và thảo luận nhóm các phương pháp, công cụ sau để biết chức năng, tác dụng của từng công cụ trong công việc kiểm thử chương trình.



Có rất nhiều phương pháp và công cụ khác nhau để kiểm thử chương trình. Các công cụ này không những có mục đích *tìm ra lỗi* (hay bug) của chương trình mà còn có tác dụng *phòng ngừa* và *ngăn chặn* các lỗi phát sinh tiếp trong tương lai.

a) Quan sát mã lỗi Runtime và bắt lỗi ngoại lệ

Nếu chương trình có lỗi Runtime (tức là đang chạy bị dừng lại), cần quan sát các mã lỗi (mã lỗi ngoại lệ) để kiểm tra vị trí dòng lệnh sinh ra lỗi này. Từ đó phân tích, tìm và sửa lỗi.

b) Kiểm thử chương trình với các bộ dữ liệu test

Chương trình cần được thử với một số bộ dữ liệu test gồm đầu vào tiêu biểu phụ thuộc đặc thù của bài toán và kết quả đầu ra đã biết trước. Các bộ test có thể có đầu vào theo các tiêu chí khác nhau như độ lớn và tính đa dạng của dữ liệu. Cần chú ý một số điểm sau:

- **Cần có nhiều bộ test** (theo các tiêu chí khác nhau như độ lớn, tính đa dạng của dữ liệu,...).
- **Cần có bộ test ngẫu nhiên**. Việc sinh ngẫu nhiên dữ liệu đầu vào trong miền xác định của chương trình làm tăng khả năng tìm lỗi nếu có.
- **Cần có bộ test dữ liệu ở vùng biên**. Ví dụ dữ liệu đầu vào là cặp (x, y) xác định trên miền $0 \leq x, y \leq 1$. Khi đó cần kiểm tra chương trình với bộ dữ liệu biên là $(0; 0)$, $(0; 1)$, $(1; 0)$ và $(1; 1)$. Thực tế cho thấy thường phát sinh lỗi tại các vùng biên hoặc lân cận của biên. Một ví dụ khác của dữ liệu biên là cần tìm các bộ test với n và các giá trị $(x_1, x_2, \dots, x_n)$ rất lớn (vùng cận biên lớn).

c) In các thông số trung gian

Bổ sung vào giữa các dòng lệnh các lệnh `print()` để in ra các biến trung gian, qua đó kiểm tra các quy trình hay thuật toán được viết có đúng không.

Giả sử chương trình có đầu vào là $(x_1, x_2, \dots, x_n)$, đầu ra là $(a_1, a_2, \dots, a_m)$ nhưng có sử dụng các biến trung gian $(y_1, y_2, \dots, y_k)$. Khi đó với mỗi bộ test đầu vào, chúng ta sẽ bổ sung vào các dòng lệnh của chương trình để in ra các giá trị trung gian:

$$(x_1, x_2, \dots, x_n), (y_1, y_2, \dots, y_k), (a_1, a_2, \dots, a_m).$$

Thông qua các giá trị trung gian trong quá trình thực hiện chương trình, nếu kết quả cuối cùng có lỗi thì sẽ dễ tìm ra lỗi đó.

d) Sử dụng công cụ break point (điểm dừng)

Công cụ break point cho phép tạo ra các "điểm dừng" bên trong chương trình. Khi chạy, chương trình sẽ tạm dừng lại tại các "điểm dừng" cho phép người kiểm thử có thể quan sát các thông tin khác bên trong chương trình, qua đó kiểm tra tính đúng đắn của chương trình.

Trên thực tế sử dụng phương pháp điểm dừng thường kết hợp với phương pháp in các giá trị trung gian sẽ là hiệu quả hơn để kiểm thử chương trình.

Một số ghi nhớ:

- Sử dụng công cụ in các biến trung gian.
- Sử dụng công cụ sinh các bộ dữ liệu test.
- Sử dụng công cụ điểm dừng trong phần mềm soạn thảo lập trình.
- Quan sát các mã lỗi của chương trình nếu phát sinh.

2. VÍ DỤ MINH HOẠ



Xét ví dụ sau: Nhập từ bàn phím hai số tự nhiên m, n , tính ƯCLN của hai số này.

Gọi $\text{gcd}(m, n)$ là ƯCLN của hai số tự nhiên m, n . Thuật toán của bài toán này dựa trên thuật toán sau:

- (1) $\text{gcd}(m, m) = m$.
- (2) Nếu $n > m$ thì $\text{gcd}(m, n) = \text{gcd}(m, n-m)$.
- (3) Nếu $n < m$ thì $\text{gcd}(m, n) = \text{gcd}(m-n, n)$.

Phần cơ bản nhất của chương trình sẽ là một vòng lặp `while`, vòng lặp sẽ kết thúc khi $m = n$. Chương trình như sau:

```
1 # Tính ƯCLN của m, n
2 m = int(input("Nhập số tự nhiên m: "))
3 n = int(input("Nhập số tự nhiên n: "))
4 while m != n:
5     if m < n:
6         n = n - m
7     else:
8         m = m - n
9 print("Đáp số:", m)
```

Chúng ta sẽ tiến hành kiểm thử chương trình này. Cần tập trung kiểm tra kĩ khối lệnh của lệnh lặp **while**.

Cách 1: In ra các giá trị trung gian để kiểm soát chương trình.

Bổ sung biến **k** và hai lệnh **print()** vào chương trình như mô tả như sau:

```
# Tính ƯCLN của m, n
m = int(input("Nhập số tự nhiên m: "))
n = int(input("Nhập số tự nhiên n: "))
k = 0
while m != n:
    k = k + 1
    print("Vòng lặp", k, ":", m, n)
    if m < n:
        n = n - m
    else:
        m = m - n
print("Kết thúc vòng lặp:", m, n)
print("Đáp số:", m)
```

Bổ sung thêm biến **k** và hai lệnh **print()** để in các giá trị trung gian **k, m, n**.

Kết quả thực hiện chương trình trên như sau:

```
Nhập số tự nhiên m: 20
Nhập số tự nhiên n: 16
Vòng lặp 1 : 20 16
Vòng lặp 2 : 4 16
Vòng lặp 3 : 4 12
Vòng lặp 4 : 4 8
Kết thúc vòng lặp: 4 4
Đáp số: 4
```

Quan sát sự thay đổi của giá trị các biến **k, m, n** trong quá trình thực hiện chương trình để phát hiện lỗi (nếu có), đồng thời hiểu được lỗi này và tìm cách sửa lỗi.

Cách 2: Sử dụng công cụ tạo điểm dừng của phần mềm soạn thảo lập trình.

Thiết lập điểm dừng tại dòng 4 của chương trình như hình sau. Đây là vị trí bắt đầu chuẩn bị vào vòng lặp.

```
1 # Tính ƯCLN của m, n
2 m = int(input("Nhập số tự nhiên m: "))
3 n = int(input("Nhập số tự nhiên n: "))
4 while m != n:
5     if m < n:
6         n = n - m
7     else:
8         m = m - n
9     print("Đáp số:", m)
```

Thiết lập điểm dừng tại dòng 4 của chương trình, đây là vị trí bắt đầu một vòng lặp mới của lệnh **while**.

Khi chạy chương trình sẽ dừng lại trước mỗi vòng lặp, chúng ta sẽ ghi lại các giá trị m, n vào một bảng như bảng sau. Khi kết thúc hết vòng lặp thì kết quả chương trình chính là giá trị m.

Vòng lặp	m	n	Kết quả
1	20	16	
2	4	16	
3	4	12	
4	4	8	
Kết thúc lặp	4	4	4

Cả hai cách để kiểm soát lỗi là in các giá trị trung gian và thiết lập điểm dừng đều hiệu quả.



LUYỆN TẬP

1. Chương trình của em khi chạy phát sinh lỗi ngoại lệ ZeroDivisionError. Đó là lỗi gì và em sẽ xử lý lỗi này như thế nào?

2. Chương trình sau có lỗi không? Nếu có thì tìm và sửa lỗi.

```
m = input("Nhập số tự nhiên m: ")
n = input("Nhập số tự nhiên n: ")
print("Tổng hai số đã nhập là:", m+n)
```



VẬN DỤNG

1. Chương trình sau có chức năng sắp xếp một dãy số cho trước. Hãy kiểm tra xem chương trình có lỗi không. Nếu có thì tìm và sửa lỗi.

```
A = [10,1,5,2,8,0,4]
for i in range(len(A)-1):
    j = i
    while j > 1 and A[j] < A[j-1]:
        A[j],A[j-1] = A[j-1],A[j]
        j = j - 1
print(A)
```

2. Để kiểm thử một chương trình, nếu chỉ bằng việc kiểm tra thông qua các bộ dữ liệu test thì có bảo đảm tìm ra hết lỗi của chương trình hay không? Vì sao?

SAU BÀI NÀY EM SẼ:

- Thực hành viết chương trình đơn giản bằng ngôn ngữ Python.
- Thực hành được các bước gỡ rối chương trình bằng công cụ debug – thiết lập điểm dừng và chạy theo từng lệnh.

Nhiệm vụ 1. Viết chương trình nhập từ bàn phím số tự nhiên n , kiểm tra n có phải là số nguyên tố hay không. Nếu n là hợp số thì in ra kết quả phân tích n thành tích các thừa số nguyên tố. Chú ý số 1 không là nguyên tố và cũng không là hợp số.

Hướng dẫn. Sử dụng biến danh sách NT để lưu các thừa số nguyên tố của n . Chương trình sẽ thiết lập danh sách NT chỉ khi $n > 1$. Kết quả của chương trình sẽ như sau:

- Nếu $n = 1$ thì danh sách NT sẽ rỗng.
- Nếu $n > 1$ thì danh sách NT không rỗng. Độ dài danh sách $\text{len}(\text{NT})$ sẽ bằng 1 khi và chỉ khi n là số nguyên tố.

Nếu $\text{len}(\text{NT}) > 1$ thì chương trình sẽ in ra khai triển n thành tích các thừa số nguyên tố, khai triển này sẽ có dạng: $n = p_1 \times p_1 \times \dots \times p_k$.

phantichnt.py

```
# Nhập số tự nhiên từ bàn phím và kiểm tra n có phải là số nguyên tố hay không.
# Nếu n = 1 thì thông báo n không phải là số nguyên tố.
# Nếu n là hợp số thì in ra phân tích n thành tích các thừa số nguyên tố.
n = int(input("Nhập số tự nhiên n: "))
m = n
k = 2
NT = []
while m > 1:
    while m%k != 0:
        k = k + 1
    NT.append(k)
    m = m//k
count = len(NT)
if count == 0:
    print(n, "không là số nguyên tố")
elif count == 1:
    print(n, "là số nguyên tố")
else:
    print(n, "là hợp số")
    print(n, "=", end = " ")
    for i in range(count):
        if i < count - 1:
            print(NT[i], "x", end = " ")
        else:
            print(NT[i])
```

Chạy chương trình với công cụ gỡ lỗi của phần mềm lập trình. Thiết lập một điểm dừng tại dòng 20 của chương trình như sau:

```

13     m = n
14     k = 2
15     NT = []
16     while m > 1:
17         while m%k != 0:
18             k = k + 1
19             NT.append(k)
20         m = m//k
21     count = len(NT)

14     k = 2
15     NT = []
16     while m > 1:
17         while m%k != 0:
18             k = k + 1
19             NT.append(k)
20         m = m//k
21     count = len(NT)
22     if count == 0:
23         print(n,"không là số nguyên tố")
24     elif count == 1:
25         print(n,"là số nguyên tố")
26     else:

```

Điểm dừng của chương trình được đặt trước lệnh $m = m/k$, sau khi k là ước số nguyên tố tiếp theo được phát hiện và đưa vào danh sách NT. Quá trình gỡ lỗi được tiến hành để kiểm tra sự thay đổi các biến n, m, k có đúng theo thuật toán hay không.

Khi chạy, chương trình sẽ chạy và dừng lại trước điểm dừng (trên màn hình dòng 20 được đánh dấu). Nháy nút để chạy tiếp chương trình.

Mỗi lần chương trình dừng lại có thể quan sát các biến n, m, k để kiểm tra tính đúng đắn của chương trình.

Thiết lập bảng theo dõi các giá trị trung gian k, m, n, NT sẽ như sau, giả sử giá trị nhập ban đầu của $n = 100$:

k	m	n	NT	Kết thúc
2	100	100	[2]	
2	50	100	[2, 2]	
5	25	100	[2, 2, 5]	
5	5	100	[2, 2, 5, 5]	
				Thông báo: $100 = 2 \times 2 \times 5 \times 5$

Nhiệm vụ 2. Viết chương trình nhập từ bàn phím ba số thực a, b, c và tìm nghiệm của phương trình bậc hai: $ax^2 + bx + c = 0$.

Chương trình cần xét đầy đủ các trường hợp xảy ra.

Hướng dẫn.

Với bộ dữ liệu a, b, c đã nhập (là các số thực), chúng ta cần xét đầy đủ các trường hợp sau:

- Nếu $a = b = c = 0$ phương trình có vô số nghiệm.
- Nếu $a = b = 0; c \neq 0$, phương trình vô nghiệm.
- Nếu $a = 0; b \neq 0$ phương trình là bậc nhất và có nghiệm duy nhất.
- Nếu $a \neq 0$, giải phương trình bậc hai. Nghiệm sẽ phụ thuộc vào giá trị $\Delta = b^2 - 4ac$. Phương trình vô nghiệm, có một nghiệm kép hoặc hai nghiệm phân biệt phụ thuộc vào giá trị Δ là nhỏ hơn 0, bằng 0 hay lớn hơn 0.

Chương trình được thiết kế thông qua các hàm sau:

- `NhapDL()`: hàm nhập ba số a, b, c từ bàn phím.
- `GiaiPT1(b, c)`: hàm giải phương trình bậc nhất: $bx + c = 0$.
- `GiaiPT2(a, b, c)`: hàm giải phương trình bậc hai: $ax^2 + bx + c = 0$.

Trong bài thực hành chúng ta sử dụng cấu trúc mở rộng của lệnh rẽ nhánh `if ... else` trong Python khi các lệnh này lồng nhau. Khi đó các lệnh rẽ nhánh lồng nhau trong mô hình bên trái sẽ được viết gọn hơn như mô hình bên phải.

<pre>if <điều kiện 1>: <nhóm lệnh 1> else: if <điều kiện 2>: <nhóm lệnh 2> else: <nhóm lệnh 3></pre>	<pre>if <điều kiện 1>: <nhóm lệnh 1> elif <điều kiện 2>: <nhóm lệnh 2> else: <nhóm lệnh 3></pre>
--	--

Chú ý: Cấu trúc `if ... elif .... else` có thể lồng nhau nhiều lần.

Chương trình đầy đủ như sau:

giaipt.py

```
# Nhập từ bàn phím ba số thực a, b, c và tìm nghiệm của phương trình
#  $ax^2 + bx + c = 0$ .
```

```
def sqrt(x):
    return x**0.5
```

```
def NhapDL():
    s = input("Nhập ba số a, b, c cách nhau bởi dấu cách: ")
    snum = s.split()
    return float(snum[0]), float(snum[1]), float(snum[2])
```

```
def GiaiPT1(b,c):
    if b != 0:
        print("Phương trình có một nghiệm duy nhất:", round(-c/b,1))
    elif c == 0:
        print("Phương trình có vô số nghiệm")
    else:
        print("Phương trình vô nghiệm")
```

```

def GiaiPT2(a,b,c):
    if a == 0:
        GiaiPT1(b,c)
    else:
        delta = b*b - 4*a*c
        if delta > 0:
            x1 = (-b + sqrt(delta))/(2*a)
            x2 = (-b - sqrt(delta))/(2*a)
            print("Phương trình có hai nghiệm khác biệt")
            print("x1 =", round(x1,1), "x2 =", round(x2,1))
        elif delta == 0:
            x = (-b / (2*a))
            print("Phương trình có nghiệm kép")
            print("x1,2 = ", round(x,1))
        else:
            print("Phương trình vô nghiệm")
# Chương trình chính
a,b,c = NhapDL()
GiaiPT2(a,b,c)

```



LUYỆN TẬP

- Viết chương trình yêu cầu nhập số thực dương a. Chương trình cần kiểm tra dữ liệu nhập như sau: Nếu số đã nhập nhỏ hơn hoặc bằng 0 thì thông báo: "Nhập sai, số a phải lớn hơn 0. Hãy nhập lại". Chương trình chỉ dừng sau khi người dùng nhập đúng.
- Viết chương trình in bảng cửu chương ra màn hình như sau:
 - Hàng thứ nhất in ra bảng nhân 1, 2, 3, 4, 5.
 - Hàng thứ hai in ra bảng nhân 6, 7, 8, 9, 10.



VẬN DỤNG

- Viết chương trình nhập hai số tự nhiên Y1, Y2 là số năm, $Y2 > Y1$. Tính xem trong khoảng thời gian từ năm Y1 đến năm Y2 có bao nhiêu năm nhuận. Áp dụng tính xem trong thế kỉ XXI có bao nhiêu năm nhuận.
- Gọi $U'CLN(a, b)$ là hàm $U'CLN$ của hai số tự nhiên a, b. Để thấy ta có $U'CLN(a, b) = U'CLN(b, a \% b)$ nếu $b > 0$ và $U'CLN(a, 0) = a$. Từ đó hãy viết chương trình nhập hai số a, b và tính $U'CLN$ của a và b.

SAU BÀI NÀY EM SẼ:

- Thực hành ôn tập lập trình Python.
- Thực hành lập trình giải bài toán có tính liên môn.

Nhiệm vụ 1. Viết chương trình nhập họ tên đầy đủ từ bàn phím, ví dụ "Nguyễn Thị Mai Hương", sau đó tách riêng phần tên, họ, đệm và in ra màn hình.

Hướng dẫn. Trong Bài 25 chúng ta sẽ biết cách tách phần họ đệm và tên từ một chuỗi họ và tên. Bài này yêu cầu thêm tách phần đệm, tức là phần nằm giữa tên và họ ra bằng cách sử dụng lệnh `join()`. Chuỗi ký tự ban đầu được tách thành một danh sách dùng hàm `split()`. Sau khi lấy phần họ và tên, phần đệm sẽ lấy ra theo lệnh sau:

```
dem = " ".join(slist[1:n-1])
```

Trong đó `slist` là danh sách được tách ra từ chuỗi ban đầu, `n` là độ dài của chuỗi `slist`.

Nhập, chạy thử và kiểm tra chương trình sau:

```
hoten = input("Nhập họ tên đầy đủ của em: ")
slist = hoten.split()
n = len(slist)
ho = slist[0]
ten = slist[n-1]
dem = " ".join(slist[1:n-1])
print("Tên của em là",ten)
print("Họ của em là",ho)
if n > 2:
    print("Đệm của em là",dem)
```

Nhiệm vụ 2. Trọng lượng của em trên các hành tinh khác.

Chương trình yêu cầu nhập trọng lượng của em (tính theo đơn vị N – Newton) trên Trái Đất và tính trọng lượng của em trên một hành tinh khác (ví dụ Mặt Trăng, Hỏa tinh, Kim tinh, Thổ tinh, Mộc tinh, Mặt Trời).

Hướng dẫn. Trọng lượng đo lực hút của Trái Đất (hay hành tinh) lên vật thể. Trọng lượng có đơn vị đo N (Newton). Khối lượng vật thể tính bằng kg và giá trị này không thay đổi. Chúng ta có công thức:

$$P = m \times g \quad (1)$$

Trong đó P là trọng lượng tính bằng N , m là khối lượng tính bằng kg , g là gia tốc trọng trường của Trái Đất (hay hành tinh), tính theo m/s^2 . Trên Trái Đất, $g = 9.8 m/s^2$. Trên mỗi hành tinh các giá trị g sẽ khác nhau. Danh sách các hành tinh được lưu trong biến `planet`, các trọng lực tương ứng lưu trong danh sách `gravities`.

Biết trọng lượng của một người trên Trái Đất (ví dụ P_0) thì sẽ dễ dàng tính được trọng lượng của người này trên một hành tinh khác nếu biết giá trị g của hành tinh đó. Gọi P là trọng lượng cần tìm, khi đó ta có công thức sau, suy trực tiếp từ công thức (1).

$$m = P_0/9.8 = P/g, \text{ vậy suy ra } P = P_0 \times g/9.8. \quad (2)$$

Em hãy nhập chương trình sau và kiểm tra tính đúng đắn của chương trình.

weight.py

```
def string(w):
    s = ""
    for i in range(len(w)):
        s = s+str(i+1)+". "+w[i] + " "
    return s

planet = ["Mặt Trăng", "Hoả tinh", "Kim tinh", "Mộc tinh", "Thổ tinh", "Mặt Trời"]
gravities = [1.62, 3.711, 8.83, 24.79, 10.44, 274.0]
P_earth = float(input("Nhập trọng lượng của em, tính theo N: "))
k = int(input("Nhập số thứ tự hành tinh\n"+string(planet)+": "))
Grp = gravities[k-1]
P = round(P_earth*Grp/9.8,3)
print("Trọng lượng của em trên", planet[k-1], "là:", P, "N")
```

Nhiệm vụ 3. Kiểm tra tính hợp lệ của ba tham số ngày, tháng, năm.

Chương trình sẽ yêu cầu nhập ba số tự nhiên: ngày, tháng, năm từ bàn phím theo khuôn dạng, ví dụ nhập 08-02-2021. Chương trình sẽ thông báo bộ dữ liệu đã nhập là hợp lệ hay không hợp lệ.

Hướng dẫn. Bộ dữ liệu chính cần nhập sẽ đặt tên là **day**, **month**, **year**. Nhiệm vụ của bài toán là nhập bộ dữ liệu này và kiểm tra tính hợp lệ theo các yêu cầu về lịch của ngày, tháng, năm.

Điểm đặc biệt nhất cần chú ý là kiểm tra năm **year** có phải là nhuận không, nếu là nhuận thì tháng 2 phải có 29 ngày so với các năm không nhuận tháng 2 có 28 ngày. Chúng ta sử dụng biến danh sách số **thang** để lưu số ngày của các tháng trong năm. Sau mỗi lần nhập ba số **day**, **month**, **year** cần kiểm tra năm nhuận để cập nhật tháng 2. Khi đó, chương trình kiểm tra có thể viết đơn giản như sau:

```

thang = [31,28,31,30,31,30,31,31,30,31,30,31]

def nhuan(year):
    return year%400 == 0 or (year%4 == 0 and year%100 != 0)

date = input("Nhập thời gian theo dạng ngày - tháng - năm: ")
tg = date.split("-")
day,month,year = int(tg[0]),int(tg[1]),int(tg[2])

if nhuan(year):
    thang[1] = 29
else:
    thang[1] = 28

if year > 0 and 1 <= month <= 12 and 1<= day <= thang[month-1]:
    print(day,"-",month,"-",year, "là hợp lệ")
else:
    print("Bộ dữ liệu đã nhập không hợp lệ")

```



VẬN DỤNG

- Trong các phần mềm bảng tính điện tử, dữ liệu ngày tháng được coi là số ngày tính từ ngày 1-1-1990. Viết chương trình:
 - Nhập số tự nhiên n từ bàn phím và tính xem số đó ứng với ngày, tháng, năm nào.
 - Nhập thời gian theo khuôn dạng ngày - tháng - năm (ví dụ 8-10-2021), tính số ngày ứng với ngày này theo phần mềm bảng tính điện tử.
- Mở rộng bài tập trong phần luyện tập như sau:
 - Việc sắp xếp thứ tự phải ưu tiên tính theo tên trước, rồi đến họ, rồi đến đệm.
 - Sắp xếp theo thứ tự của bảng chữ cái tiếng Việt.

Chú ý: Bảng chữ cái tiếng Việt (bao gồm cả dấu thanh) được sắp xếp theo thứ tự sau: AAAAAÁÀẢÃĂẰẴB C D Đ E Ê Ë Ė È É Ě Ğ H I J K L M N O Ò Ó Ô Õ Ö Ø Ù Ú Û Ü Ý Þ ß à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ø ù ú û ü ý þ ÿ.

- 155